

Contents

Problem 1.	4
Problem 1. Solutions	6
0.0.1 Method One	7
0.0.2 Method Two	9
0.0.3 Method 3	13
Problem 2.	21
Problem 2. Solutions	24
Appendix I: Problem 2. Codes	43

List of Figures

1	Feed back control diagram for a heat exchanger	4
2	Method 1 vs. True Gain	18
3	Method 1 + 20% Increase vs. True Gain	18
4	Method 1 - 20% Decrease vs. True Gain	18
5	Method 2 vs. True Gain	19
6	Method 2 + 20% Increase vs. True Gain	19
7	Method 2 - 20% Decrease vs. True Gain	19
8	Method 3 vs. True Gain	20
9	Method 3 + 20% Increase vs. True Gain	20
10	Method 3 - 20% Decrease vs. True Gain	20
11	Feed back control diagram for a heat exchanger	21
12		21
13		22
14	Problem 2: Part 8 - Regulatory Control	29
15	Problem 2: Part 9 - Servo Control	30
16	Problem 2: Part 10; Regulatory Control With Time Delay	31
17	Problem 2: Part 10; Servo Control With Time Delay	32
18	Process diagram of disturbance after process	32
19	Process Diagram of disturbance before process	33

20	Problem 2: Part 10; Ziegler-Nichols Method:Regulatory Control With Time Delay	37
21	Problem 2: Part 10; Ziegler-Nichols Method:Servo Control With Time Delay . . .	38
22	Problem 2: Part 10; Ziegler-Nichols Ultimate Gain Method:Regulatory Control With Time Delay	39
23	Problem 2: Part 10; Ziegler-Nichols Ultimate Gain Method:Servo Control With Time Delay	39
24	Problem 2: Part 10; Cohen-Coon Method:Regulatory Control With Time Delay .	40
25	Problem 2: Part 10; Cohen-Coon Method:Servo Control With Time Delay	41

PROBLEM 1.

Consider the feed back control block diagram for a process heat exchanger depicted in the Figure Below.

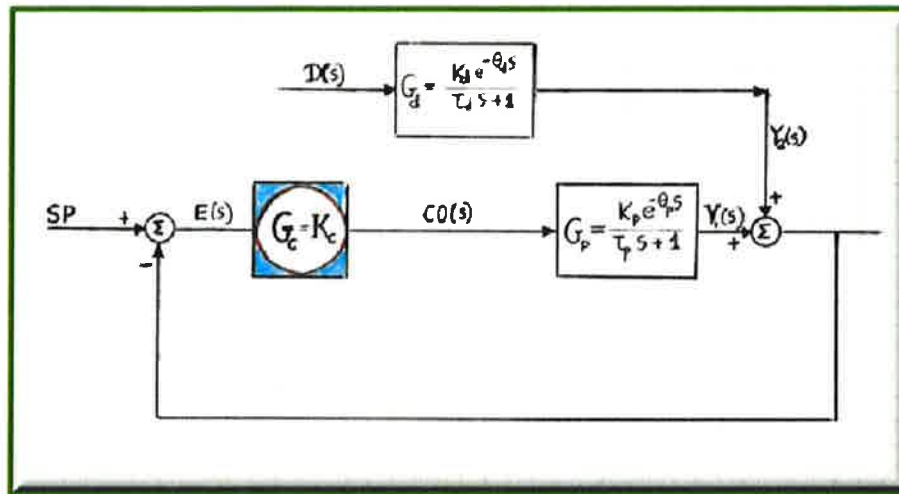


Figure 1: Feed back control diagram for a heat exchanger

Derive an expression for the controller gain K_C as a function of the process gain K_p , process lag τ_p , and process time delay θ_p , that correspond to critical damping of the process response. You need to devise two different ways to derive these controller gain functions. Clearly show all the steps in your derivations and place the final formulas for K_C in a rectangular shape.

Calculate the controller gain using the two formulas that you have developed for a process with the following characteristics: $\{K_p=6.73, \theta_p=8.47, \tau_p=25.0\}$. Now perform a simulation using the PIDController.xlsm modeling tool for a set point step change equal to $SP=10.0$. Using a trial and error process calculate the true controller gain that corresponds to critical damping and then calculate error in your estimation via the two different methods. Include the following plots in your final report:

1. Process response for a set point step change equal to $SP=10.0$ with the true gain corresponding to critical damping vs. response using gain from your first devised method.

2. Process response for a set point step change equal to $SP=10.0$ with the true gain corresponding to critical damping vs. response using gain from your second devised method.
3. Process response for a set point step change equal to $SP=10.0$ with the gain you estimated from the first devised method, but increased to a new level =20% higher
4. Process response for a set point step change equal to $SP=10.0$ with the gain that you estimated from the second devised method, but increased to a new level -20% higher

Explain why the process response for the last two simulations is under-damped, over-damped, critically damped, or unstable.

PROBLEM 1. SOLUTIONS

Transfer Function:

$$PV(s) = G_P * (G_C * (SP(s) - PV(s)) + G_d * D(s)) \quad (1)$$

$$PV(s) = G_P * G_C * SP(s) - G_P * G_C * PV(s) + G_P * G_d * D(s) \quad (2)$$

$$PV(s) + G_P * G_C * PV(s) = G_P * G_C * SP(s) + G_P * G_d * D(s) \quad (3)$$

$$PV(s)(1 + G_P * G_C) = G_P * G_C * SP(s) + G_P * G_d * D(s) \quad (4)$$

$$PV(s) = \frac{G_P * G_C * SP(s) + G_P * G_d * D(s)}{1 + G_P * G_C} \quad (5)$$

When...

$$G_C = K_C \quad (6)$$

$$G_P = \frac{K_P * e^{-\theta_P * s}}{\tau_P * s + 1} \quad (7)$$

$$G_d = \frac{K_d * e^{-\theta_d * s}}{\tau_d * s + 1} \quad (8)$$

Then...

$$PV(s) = \frac{\frac{K_P * e^{-\theta_P * s}}{\tau_P * s + 1} * K_C * SP(s) + \frac{K_P * e^{-\theta_P * s}}{\tau_P * s + 1} * \frac{K_d * e^{-\theta_d * s}}{\tau_d * s + 1} * D(s)}{1 + \frac{K_P * e^{-\theta_P * s}}{\tau_P * s + 1} * K_C} \quad (9)$$

Closed Loop Transfer Function:

$$0 = 1 + \frac{K_P * e^{-\theta_P * s}}{\tau_P * s + 1} * K_C \quad (10)$$

0.0.1 Method One

1:1 Pade Approximation

$$e^{-s\theta} = \frac{1 - \frac{s\theta}{2}}{1 + \frac{s\theta}{2}} \quad (11)$$

Using the Pade Approximation...

$$e^{-\theta_p S} = \frac{1 - \frac{S\theta_p}{2}}{1 + \frac{S\theta_p}{2}} \quad (12)$$

Plugging back into the closed loop transfer function

$$0 = 1 + K_P K_C \left(\frac{1 - \frac{S\theta_p}{2}}{1 + \frac{S\theta_p}{2}} \right) \left(\frac{1}{\tau_P S + 1} \right) \quad (13)$$

$$0 = (\tau_P S + 1) \left(1 + \frac{S\theta_p}{2} \right) + K_P K_C \left(1 - \frac{S\theta_p}{2} \right) \quad (14)$$

$$0 = \tau_P S + \tau_P S^2 \frac{\theta_p}{2} + \frac{S\theta_p}{2} + 1 + K_P K_C - K_P K_C S \frac{\theta_p}{2} \quad (15)$$

$$0 = S^2 \tau_P \frac{\theta_p}{2} + S \left(\tau_P + \frac{\theta_p}{2} - K_P K_C \frac{\theta_p}{2} \right) + 1 + K_P K_C \quad (16)$$

Dividing the whole equation by

$$1 + K_P K_C \quad (17)$$

$$0 = S^2 \frac{\tau_P \theta_p}{2(1 + K_P K_C)} + S \left(\frac{\tau_P + (0.5\theta_p) - K_P K_C (0.5\theta_p)}{(1 + K_P K_C)} \right) + 1 \quad (18)$$

This equation is similar to the second order closed loop transfer function shown below

Closed Loop Second Order Transfer Function

$$0 = \tau^2 S^2 + 2\zeta \tau S + 1 \quad (19)$$

Therefore

$$\tau_p^2 = \frac{\tau_p \theta_p}{2(1 + K_P K_C)} \quad (20)$$

$$2\zeta \tau_p = \left(\frac{\tau_p + (0.5\theta_p) - K_P K_C (0.5\theta_p)}{(1 + K_P K_C)} \right) \quad (21)$$

Both equations are solved for τ_p

$$\tau_p = \left[\frac{\tau_p \theta_p}{2(1 + K_P K_C)} \right]^{\frac{1}{2}} \quad (22)$$

$$\tau_p = \left(\frac{2\tau_p + \theta_p - K_P K_C \theta_p}{2(1 + K_P K_C)} \right) \left(\frac{1}{2\zeta} \right) \quad (23)$$

$$\left[\frac{\tau_p \theta_p}{2(1 + K_P K_C)} \right]^{\frac{1}{2}} = \left(\frac{2\tau_p + \theta_p - K_P K_C \theta_p}{2(1 + K_P K_C)} \right) \left(\frac{1}{2\zeta} \right) \quad (24)$$

For a true controller gain to correspond to critical damping ζ must equal one

ζ is set to one and the other values are plugged in to solve for K_C

$$K_P = 6.73$$

$$\theta_p = 8.47$$

$$\tau_p = 25.0$$

After using GoalSeek in Excel

$$K_C = 0.0972 \quad (25)$$

0.0.2 Method Two

Using Taylor expansion for the exponential found in the closed loop transfer function

$$e^{-\theta S} = 1 - \theta S + \frac{\theta^2 S^2}{2} - \frac{\theta^3 S^3}{6} + \frac{\theta^4 S^4}{24} - \frac{\theta^5 S^5}{120} + O(S^6) \dots \quad (26)$$

Because the second order closed loop transfer function is quadratic, the expansion is reduced to the first three terms, becoming...

$$e^{-\theta S} = 1 - \theta S + \frac{\theta^2 S^2}{2} \quad (27)$$

The expansion is plugged into the discovered closed loop transfer function in place of the exponential

$$0 = 1 + \frac{K_P(1 - \theta S + \frac{\theta^2 S^2}{2})}{\tau_P S + 1} * K_C \quad (28)$$

$$0 = \tau_P S + 1 + K_P K_C (1 - \theta S + \frac{\theta^2 S^2}{2}) \quad (29)$$

$$0 = \tau_P S + 1 + K_P K_C - K_P K_C \theta S + K_P K_C \frac{\theta^2 S^2}{2} \quad (30)$$

$$0 = S^2 \frac{K_P K_C \theta^2}{2} + S(\tau_P - K_P K_C \theta) + K_P K_C + 1 \quad (31)$$

Dividing the whole equation by

$$1 + K_P K_C \quad (32)$$

$$0 = S^2 \frac{K_P K_C \theta^2}{2(1 + K_P K_C)} + S \left(\frac{\tau_P - K_P K_C \theta}{1 + K_P K_C} \right) + 1 \quad (33)$$

This equation is similar to the second order closed loop transfer function shown below

Closed Loop Second Order Transfer Function

$$0 = \tau^2 S^2 + 2\zeta\tau S + 1 \quad (34)$$

Therefore

$$\tau_P^2 = \frac{K_P K_C \theta^2}{2(1 + K_P K_C)} \quad (35)$$

$$2\zeta\tau_P = \frac{\tau_P - K_P K_C \theta}{1 + K_P K_C} \quad (36)$$

Both equations are solved for τ_P

$$\tau_P = \left[\frac{K_P K_C \theta^2}{2(1 + K_P K_C)} \right]^{\frac{1}{2}} \quad (37)$$

$$\tau_P = \left(\frac{\tau_P - K_P K_C \theta}{1 + K_P K_C} \right) \left(\frac{1}{2\zeta} \right) \quad (38)$$

$$\left[\frac{K_P K_C \theta^2}{2(1 + K_P K_C)} \right]^{\frac{1}{2}} = \left(\frac{\tau_P - K_P K_C \theta}{1 + K_P K_C} \right) \left(\frac{1}{2\zeta} \right) \quad (39)$$

For a true controller gain to correspond to critical damping ζ must equal one

ζ is set to one and the other values are plugged in to solve for K_C

$$K_P = 6.73$$

$$\theta_P = 8.47$$

$$\tau_P = 25.0$$

After using GoalSeek in Excel

$$K_C = 0.146 \quad (40)$$

Because this approximation is so far off from the true controller gain, I decided to re-do the second method using only the first two terms of the Taylor expansion. The Taylor expansion used is shown below

$$e^{-\theta_P S} = \frac{1}{\theta_P S + 1} \quad (41)$$

The expansion is plugged into the discovered closed loop transfer function in place of the exponential

$$0 = 1 + \frac{K_P(\frac{1}{\theta_P S + 1})}{\tau_P S + 1} * K_C \quad (42)$$

$$0 = (\tau_P S + 1)(\theta_P S + 1) + K_P K_C \quad (43)$$

$$0 = S^2 \tau_P \theta_P + S(\theta_P + \tau_P) + 1 + K_P K_C \quad (44)$$

Dividing the whole equation by

$$1 + K_P K_C \quad (45)$$

$$0 = S^2 \left(\frac{\tau_P \theta_P}{1 + K_P K_C} \right) + S \left(\frac{\theta_P + \tau_P}{1 + K_P K_C} \right) + 1 \quad (46)$$

This equation is similar to the second order closed loop transfer function shown below

Closed Loop Second Order Transfer Function

$$0 = \tau^2 S^2 + 2\zeta \tau S + 1 \quad (47)$$

Therefore

$$\tau_P^2 = \frac{\tau_P \theta_P}{1 + K_P K_C} \quad (48)$$

$$2\zeta \tau_P = \frac{\theta_P + \tau_P}{1 + K_P K_C} \quad (49)$$

Both equations are solved for τ_p

$$\tau_p = \left[\frac{\tau_p \theta_p}{1 + K_p K_C} \right]^{\frac{1}{2}} \quad (50)$$

$$\tau_p = \left(\frac{\theta_p + \tau_p}{1 + K_{pC}} \right) \left(\frac{1}{2\zeta} \right) \quad (51)$$

$$\left[\frac{\tau_p \theta_p}{1 + K_p K_C} \right]^{\frac{1}{2}} = \left(\frac{\theta_p + \tau_p}{1 + K_{pC}} \right) \left(\frac{1}{2\zeta} \right) \quad (52)$$

For a true controller gain to correspond to critical damping ζ must equal one

ζ is set to one and the other values are plugged in to solve for K_C

$$K_p = 6.73$$

$$\theta_p = 8.47$$

$$\tau_p = 25.0$$

After using GoalSeek in Excel

$$K_C = 0.048 \quad (53)$$

0.0.3 Method 3

Reverse Skogestad Method

The closed loop transfer function is given below

$$0 = 1 + \frac{K_P * e^{-\theta_P * s}}{\tau_P * s + 1} * K_C \quad (54)$$

where $G_P(s)$ is equal to

$$G_P(s) = \frac{K_P * e^{-\theta_P * s}}{\tau_P * s + 1} \quad (55)$$

The idea behind Reverse Skogestad is that the process equation will be going from a 1st order equation to a 2nd order equation

In Skogestad's method, $\tau = \tau_1 + \frac{\tau_2}{2}$ and $\theta = \theta_0 + \frac{\tau_2}{2}$

For the method to be in this form, two assumptions have to be made $\tau_1 > \tau_2$ and $\theta_0 = 0$

Using the assumptions and the equations given by the Skogestad Method

$$\tau_P = \tau_1 + \frac{\tau_2}{2} \quad (56)$$

$$\theta_P = \frac{\tau_2}{2} \quad (57)$$

$$\tau_2 = 2\theta_P \quad (58)$$

$$\tau_1 = \tau_P - \theta_P \quad (59)$$

because we made the assumption that τ_1 is going to be greater than τ_2 it can be assumed that

$$\tau_P - \theta_P > 2\theta_P \quad (60)$$

which can be reduced to

$$\tau_p > 3\theta_p \quad (61)$$

In our specific problem $\tau_p=25.0$ and $\theta_p=8.47$ which means that τ_p is only 2.95 times larger than θ_p . If our assumptions are correct then this will not be a good method to use in approximating K_C but the equations for calculating K_C are given below

the new process transfer function is calculated

$$G_P(s) = \frac{K_P}{(\tau_1 S + 1)(\tau_2 S + 1)} \quad (62)$$

this function is plugged into the closed loop transfer function

$$0 = 1 + \frac{K_P}{(\tau_1 S + 1)(\tau_2 S + 1)} * K_C \quad (63)$$

$$0 = (\tau_1 S + 1)(\tau_2 S + 1) + K_P K_C \quad (64)$$

$$0 = S^2 \tau_1 \tau_2 + S(\tau_1 + \tau_2) + 1 + K_P K_C \quad (65)$$

then the entire equation is divided out by $1 + K_P K_C$

$$0 = S^2 \left(\frac{\tau_1 \tau_2}{1 + K_P K_C} \right) + S \left(\frac{\tau_1 + \tau_2}{1 + K_P K_C} \right) + 1 \quad (66)$$

This equation is similar to the second order closed loop transfer function shown below

Closed Loop Second Order Transfer Function

$$0 = \tau^2 S^2 + 2\zeta \tau S + 1 \quad (67)$$

Therefore

$$\tau_p^2 = \frac{\tau_1 \tau_2}{1 + K_P K_C} \quad (68)$$

$$2\zeta\tau_p = \frac{\tau_1 + \tau_2}{1 + K_p K_C} \quad (69)$$

Both equations are solved for τ_p

$$\tau_p = \left[\frac{\tau_1 \tau_2}{1 + K_p K_C} \right]^{\frac{1}{2}} \quad (70)$$

$$\tau_p = \left(\frac{\tau_1 + \tau_2}{1 + K_p K_C} \right) \left(\frac{1}{2\zeta} \right) \quad (71)$$

$$\left[\frac{\tau_1 \tau_2}{1 + K_p K_C} \right]^{\frac{1}{2}} = \left(\frac{\tau_1 + \tau_2}{1 + K_p K_C} \right) \left(\frac{1}{2\zeta} \right) \quad (72)$$

For a true controller gain to correspond to critical damping ζ must equal one

ζ is set to one and the other values are plugged in to solve for K_C

$$K_p = 6.73$$

$$\theta_p = 8.47$$

$$\tau_p = 25.0$$

After using GoalSeek in Excel

$$K_C = 2.815 * 10^{-5} \quad (73)$$

the equation was also solved for K_C so that it could be found if Goal Seek was not available

$$K_C = -\frac{1}{K_p} + \frac{\left(\frac{(\tau_p + \theta_p)^2}{8K_p(\tau_p - \theta_p)} \right)}{K_p} \quad (74)$$

The true controller gain found by the guess and check method on the excel file given is $K_C=0.127$

Percent error is calculated using the formula for below

$$\%Error = \frac{Theoretical - Calculated}{Theoretical} * 100 \quad (75)$$

Percent Error Method 1:

$$\%Error = \frac{0.127 - 0.0972}{0.127} * 100 = 23.46\% \quad (76)$$

Percent Error Method 2-1:

$$\%Error = \frac{0.127 - 0.146}{0.127} * 100 = -14.96\% \quad (77)$$

Percent Error Method 2-2:

$$\%Error = \frac{0.127 - 0.048}{0.127} * 100 = 62.2\% \quad (78)$$

Percent Error Method 3:

$$\%Error = \frac{0.127 - 0.00002815}{0.127} * 100 = 99.98\% \quad (79)$$

It was approximated for each graph whether the calculated K_C values was either critically damped, under damped or over damped. When determining the true K_C value, the PV before was subtracted by the current PV. When the K_C was critically damped, there would be no negatives found in these calculated meaning that there was no oscillations in the graphs. For the graphs using the other methods, there were not any found to be perfectly critically damped. Underdamped was approximated because of the "sluggishness" of the PV incline while over damped was determined because the PV graph would rise quickly, then level out.

The approximation of each is found below

- Figure 2: Method 1 - Underdamped
- Figure 3: Method 1; 20% increase- Critically damped
- Figure 4: Method 1; 20% decrease - Underdamped
- Figure 5: Method 2a - Critically damped
- Figure 6: Method 2a; 20% increase- Over damped
- Figure 7: Method 2a; 20% decrease - Underdamped
- Figure 8: Method 2b - Underdamped
- Figure 9: Method 2b; 20% increase- Under damped
- Figure 10: Method 2b; 20% decrease - Underdamped

Over all, the best method was Taylor expansion with three terms because it was the closest to being critically damped without a 20% increase or decrease

The graphs for parts I, II, III, and IV can be found below

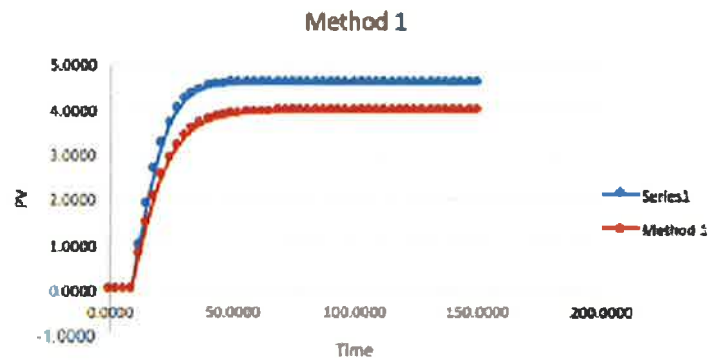


Figure 2: Method 1 vs. True Gain

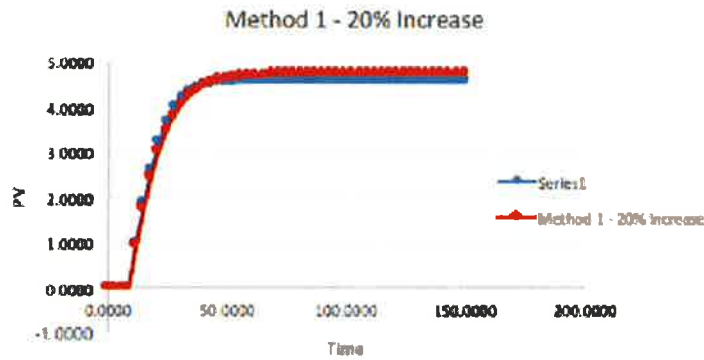


Figure 3: Method 1 + 20% Increase vs. True Gain

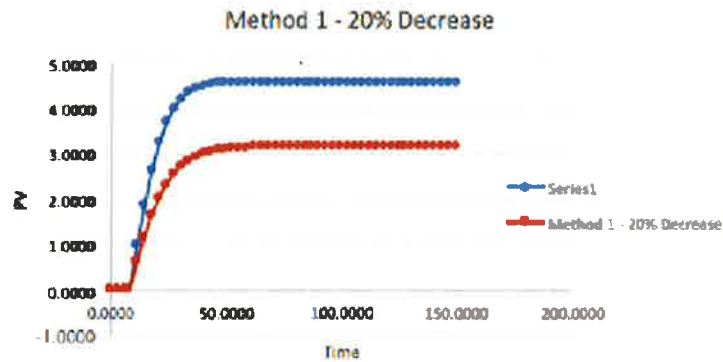


Figure 4: Method 1 - 20% Decrease vs. True Gain

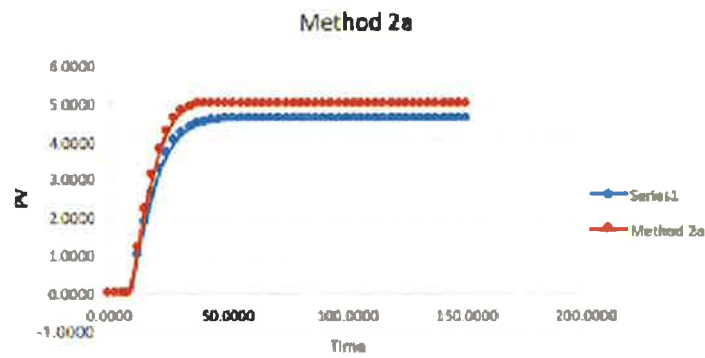


Figure 5: Method 2 vs. True Gain

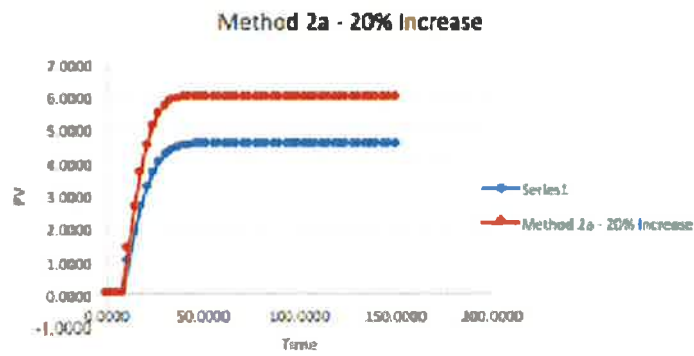


Figure 6: Method 2 + 20% Increase vs. True Gain

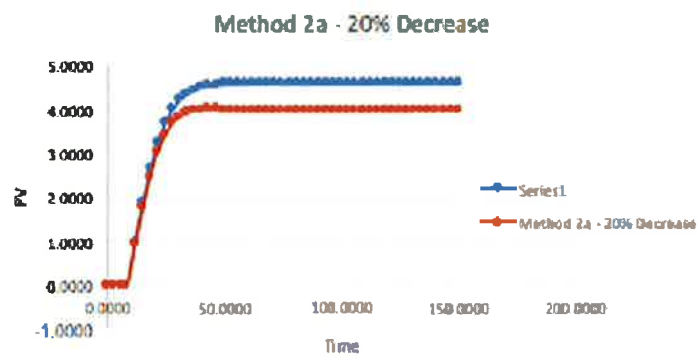


Figure 7: Method 2 - 20% Decrease vs. True Gain

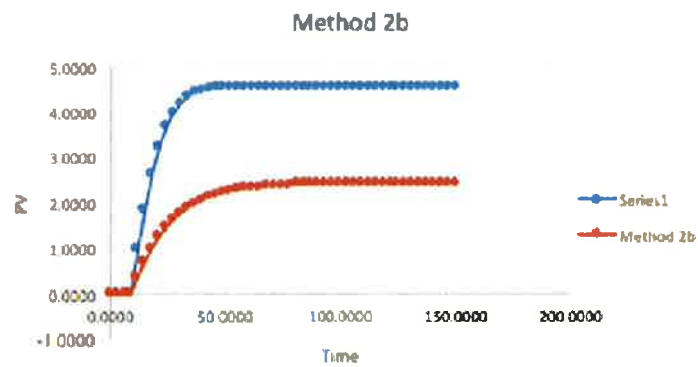


Figure 8: Method 3 vs. True Gain

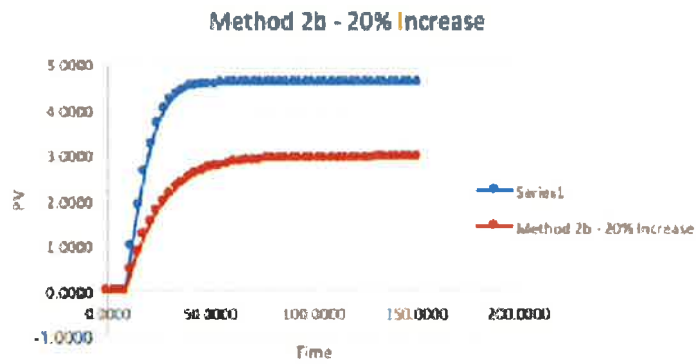


Figure 9: Method 3 + 20% Increase vs. True Gain

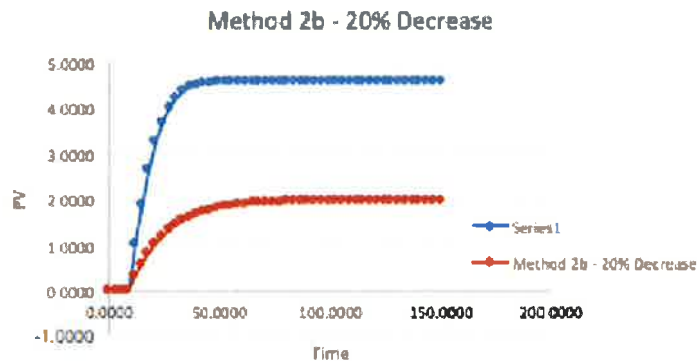


Figure 10: Method 3 - 20% Decrease vs. True Gain

PROBLEM 2.

Consider the feed back control block diagram distillation column depicted in the figure below. The transfer functions of the process have been identified with step tests and are given by the

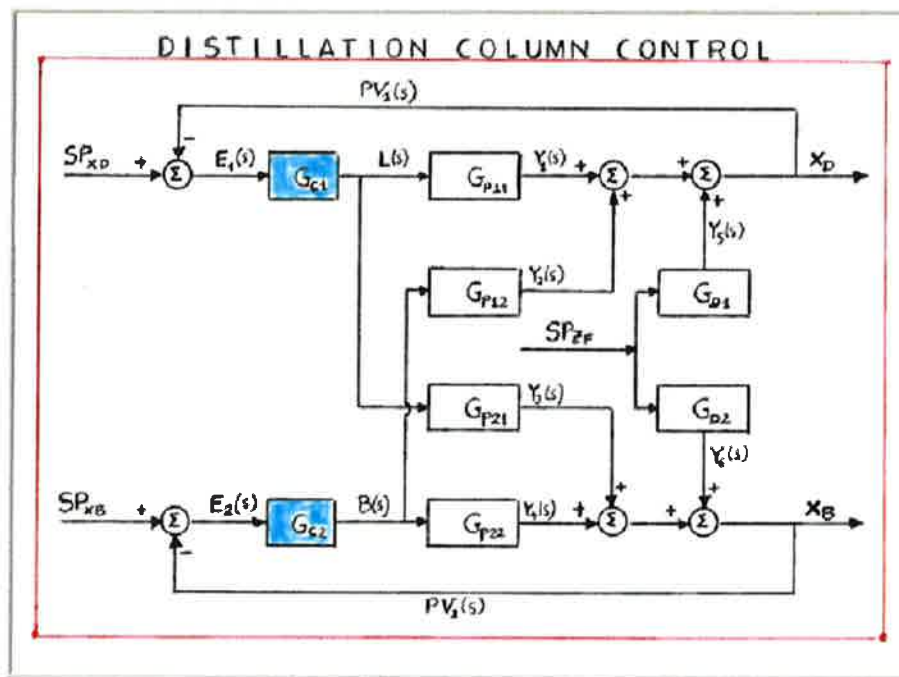


Figure 11: Feed back control diagram for a heat exchanger

following functions:

G_{P11}	G_{P12}	G_{D1}
G_{P21}	G_{P22}	G_{D2}

$$= \begin{bmatrix} \frac{0.1247}{10s+1} & \frac{0.546}{24s+1} & \frac{0.588e^{-7s}}{22s+1} \\ \frac{-0.1246}{16s+1} & \frac{1.116}{22s+1} & \frac{1.41e^{-7s}}{24s+1} \end{bmatrix}$$

Figure 12

1. Derive the differential equation for $Y_1(t)$ as a function of $L(t)$ and $B(t)$
2. Derive the differential equation for $Y_2(t)$ as a function of $L(t)$ and $B(t)$

3. Derive the differential equation for $Y_3(t)$ as a function of $L(t)$ and $B(t)$
4. Derive the differential equation for $Y_4(t)$ as a function of $L(t)$ and $B(t)$
5. Derive the differential equation for $PV_1(t)$ as a function of $L(t)$ and $B(t)$
6. Derive the differential equation for $PV_2(t)$ as a function of $L(t)$ and $B(t)$
7. If the two proposed controllers GC_1 and GC_2 are proportional-integral with controller gains $K_{C1}=32.08$ and $K_{C2}=3.58$ derive ranges of their integral time constants τ_{11} and τ_{12} for stable performance. Use the Routh criterion for this derivation.

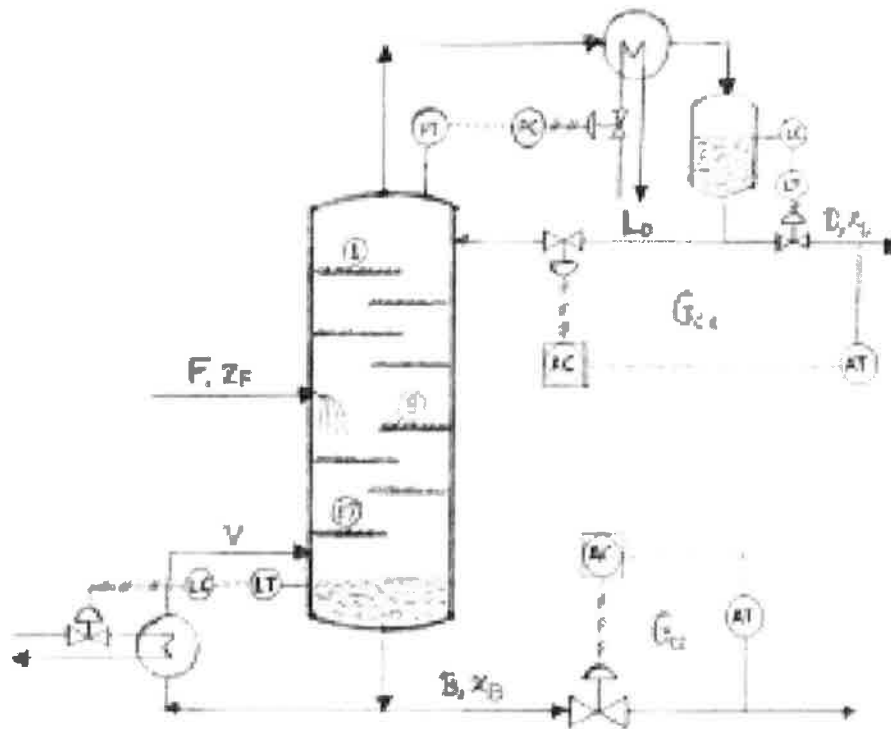


Figure 13

8. Using the figure above, perform a step change equal to 0.1 in the feed composition and examine the performance of the two controllers by plotting the responses of the distillate and bottoms composition as a function of time. Investigate whether the performance of the controllers is acceptable and if you deem necessary fine tune the controller constants. You may wish, for instance, to decrease the proportional gains and increase the integral time to eliminate the undesirable disturbances.

9. Using Figure 4, perform a step change equal to 0.02 in the distillate composition and examine the performance of the two controllers by plotting the responses of the distillate and the bottoms composition as a function of time. Investigate whether the performance of the controllers is acceptable and if you deem necessary fine tune the controller constants. You may wish, for instance, to decrease the proportional gains and increase the integral time to eliminate the undesirable disturbances.
10. Repeat parts 8 and 9 by including some dead time in the composition analyzers. In reality, this dead time can be a few minutes. Use 5.0 minutes for each composition analyzer in your simulation. Before you carry out the simulations prove that the dead time of the composition analyzer can be modeled by modifying the process transfer functions to include the a time delay in the form:

$$G_{pij} = G_{pij} * e^{s(AnalyzerDeadTime)} \quad (80)$$

For your proof of the above equation, you may use a simple feedback control block diagram with transportation delay in the measurement element. Indicate whether your proof holds for disturbance acting on the controller output or on the process variable. If your proof is not valid for one of these options what do you recommend?

Prepare plots where you indicate the performance of the distillation column control for set point tracking (servo control) and disturbance rejection (regulatory-control) with the following tuning methods

- Ziegler-Nichols
- Ziegler-Nichols Ultimate Gain
- Cohen-Coon

Describe in your own words which tuning method gives the best performance for the servo and regulatory control cases and explain the reasons for your decision.

PROBLEM 2. SOLUTIONS

1.

$$Y_1(s) = L(s) * G_{P11} \quad (81)$$

$$Y_1(s) = L(s) * \frac{0.1247}{10s + 1} \quad (82)$$

$$Y_1(s) * (10s + 1) = 0.1247 * L(s) \quad (83)$$

$$Y_1(s)(10s) + Y_1(s) = 0.1247 * L(s) \quad (84)$$

$$10 \frac{dY_1(t)}{dt} + Y_1(t) = 0.1247 * L(t) \quad (85)$$

2.

$$Y_2(s) = L(s) * G_{P12} \quad (86)$$

$$Y_2(s) = L(s) * \frac{0.546}{24s + 1} \quad (87)$$

$$Y_2(s) * (24s + 1) = 0.546 * L(s) \quad (88)$$

$$Y_2(s)(24s) + Y_2(s) = 0.546 * L(s) \quad (89)$$

$$24 \frac{dY_2(t)}{dt} + Y_2(t) = 0.546 * B(t) \quad (90)$$

3.

$$Y_3(s) = L(s) * G_{P11} \quad (91)$$

$$Y_3(s) = L(s) * \frac{-0.1246}{16s + 1} \quad (92)$$

$$Y_3(s) * (16s + 1) = -0.1246 * L(s) \quad (93)$$

$$Y_3(s)(16s) + Y_3(s) = -0.1246 * L(s) \quad (94)$$

$$16 \frac{dY_3(t)}{dt} + Y_3(t) = -0.1246 * L(t) \quad (95)$$

4.

$$Y_4(s) = L(s) * G_{P11} \quad (96)$$

$$Y_4(s) = L(s) * \frac{1.116}{22s + 1} \quad (97)$$

$$Y_4(s) * (22s + 1) = 1.116 * L(s) \quad (98)$$

$$Y_4(s)(22s) + Y_4(s) = 1.116 * L(s) \quad (99)$$

$$22 \frac{dY_4(t)}{dt} + Y_4(t) = 1.116 * B(t) \quad (100)$$

5.

$$PV_1(s) = Y_1(s) + Y_2(s) + Y_5(s) \quad (101)$$

$$Y_5(s) = SP_{ZD} * G_{D1} \quad (102)$$

$$Y_5(s) = SP_{ZD} * \frac{0.588 * e^{-7s}}{22s + 1} \quad (103)$$

$$Y_5(s)(22s + 1) = SP_{ZD} * (0.588 * e^{-7s}) \quad (104)$$

$$Y_5(s)(22s) + Y_5(s) = SP_{ZD} * (0.588 * e^{-7s}) \quad (105)$$

$$22 \frac{dY_5(t)}{dt} + Y_5(t) = 0.588 * SP_{ZD}(t - 7) \quad (106)$$

$$PV_1(s) = -10 \frac{dY_1(t)}{dt} + 0.1247 * L(t) - 24 \frac{dY_2(t)}{dt} + 0.546 * B(t) - 22 \frac{dY_5(t)}{dt} + 0.588 * SP_{ZD}(t - 7) \quad (107)$$

6.

$$PV_2(s) = Y_3(s) + Y_4(s) + Y_6(s) \quad (108)$$

$$Y_6(s) = SP_{ZD} * G_{D2} \quad (109)$$

$$Y_6(s) = SP_{ZD} * \frac{1.41 * e^{-7s}}{24s + 1} \quad (110)$$

$$Y_6(s)(24s + 1) = SP_{ZD} * (1.41 * e^{-7s}) \quad (111)$$

$$Y_6(s)(24s) + Y_6(s) = SP_{ZD} * (1.41 * e^{-7s}) \quad (112)$$

$$24 \frac{dY_6(t)}{dt} + Y_6(t) = 1.41 * SP_{ZD}(t - 7) \quad (113)$$

$$PV_2(s) = -16 \frac{dY_3(t)}{dt} - 0.1246 * L(t) - 22 \frac{dY_4(t)}{dt} + Y_4(t) = 1.116 * B(t) - 24 \frac{dY_6(t)}{dt} + 1.41 * SP_{ZD}(t - 7) \quad (114)$$

7. Closed Loop Transfer with Proportional and Integral Control

$$0 = 1 + G_{P11} G_{C1} \left(1 + \frac{1}{\tau_{11} S}\right) \quad (115)$$

$$0 = 1 + K_{C1} \left(\frac{0.1247}{10S + 1}\right) \left(1 + \frac{1}{\tau_{11} S}\right) \quad (116)$$

$$K_{C1} = 32.08 \quad (117)$$

$$0 = 1 + 32.08 \left(\frac{0.1247}{10S + 1} \right) \left(1 + \frac{1}{\tau_{11}S} \right) \quad (118)$$

$$0 = (10S + 1)(\tau_{11}S) + 32.08(0.1247)(\tau_{11}S + 1) \quad (119)$$

$$0 = 10S^2\tau_{11}S + 5\tau_{11}S + 4 \quad (120)$$

$$a_2 = 10\tau_{11} \quad (121)$$

$$a_1 = 5\tau_{11} \quad (122)$$

$$a_0 = 4 \quad (123)$$

$$\tau_{11} \geq 0 \quad (124)$$

Closed Loop Transfer with Proportional and Integral Control

$$0 = 1 + G_{P22}G_{C2} \left(1 + \frac{1}{\tau_{12}S} \right) \quad (125)$$

$$0 = 1 + K_{C1} \left(\frac{1.116}{22S + 1} \right) \left(1 + \frac{1}{\tau_{12}S} \right) \quad (126)$$

$$K_{C1} = 3.58 \quad (127)$$

$$0 = 1 + 3.58 \left(\frac{1.116}{22S + 1} \right) \left(1 + \frac{1}{\tau_{12}S} \right) \quad (128)$$

$$0 = (22S + 1)(\tau_{12}S) + 3.58(1.116)(\tau_{12}S + 1) \quad (129)$$

$$0 = 22S^2\tau_{12}S + 4.995\tau_{12}S + 3.995 \quad (130)$$

$$a_2 = 22\tau_{12} \quad (131)$$

$$a_1 = 4.995\tau_{12} \quad (132)$$

$$a_0 = 3.995 \quad (133)$$

$$\tau_{12} \geq 0 \quad (134)$$

8. Regulatory Control is defined as the process transient behavior when a disturbance enters. The idea behind regulatory control is that the control system should be able to return each controlled variable back to its set point.

It is evident that for both PV's that there is a disturbance in the system so the control overshoots the set point, then settles back out.

For this code the integral gains were set to $K_{ci1} = K_{c1}/10.0$ and $K_{ci2} = K_{c2}/22.0$

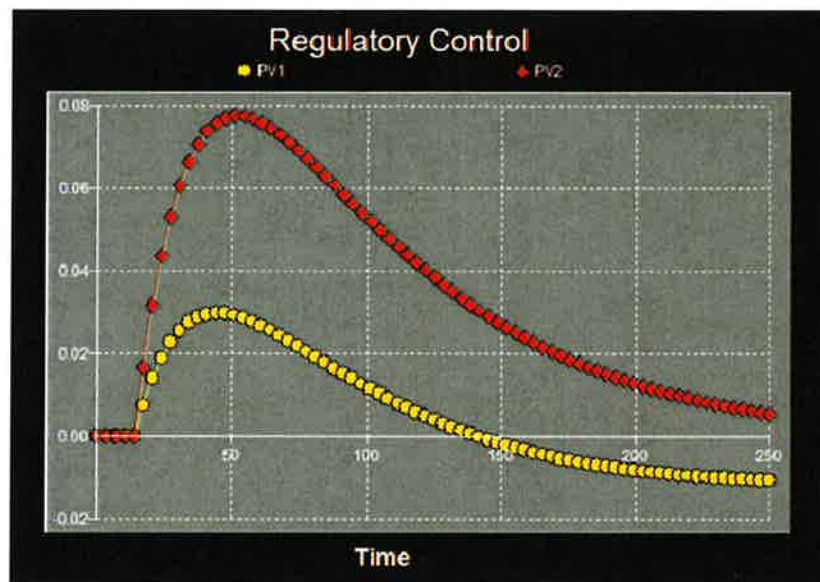


Figure 14: Problem 2: Part 8 - Regulatory Control

9. Servo Control is defined as implementing a change in the operating conditions (usually changing the set-point). In servo control, the set-point is changed and the manipulated variable is adjusted appropriately to achieve the new operating conditions.

In this graph, it is apparent that the set point change only occurred for the first controller. For PV1, the yellow line, the set point is changed so the system overshoots then settles back out at the desired set point. For PV2, the red line, there is no set point change, but there is a disturbance because the set point of the other process was changed. The system undershoots because of the disturbance and then settles back out.

For this code the integral gains were set to $K_{ci1} = K_{c1}/10.0$ and $K_{ci2} = K_{c2}/22.0$

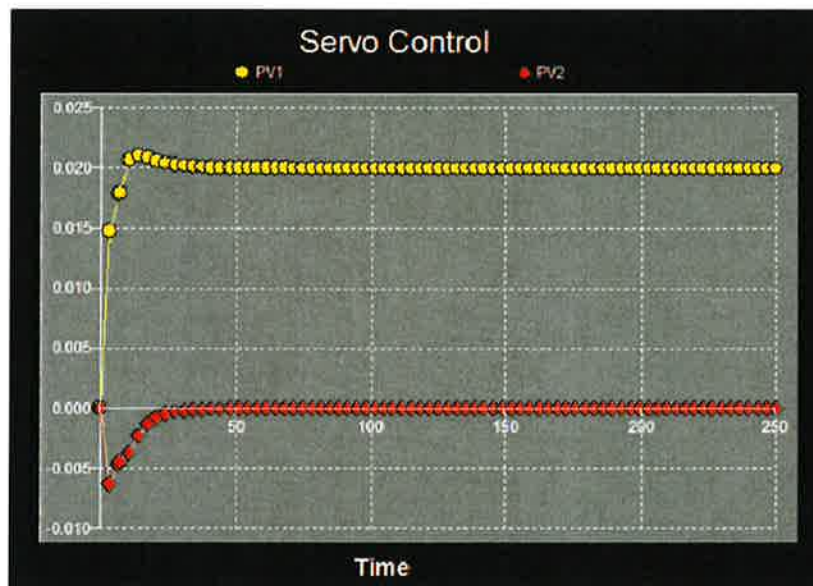


Figure 15: Problem 2: Part 9 - Servo Control

10. Part a of problem 2: part 10 is very similar to parts 8 and 9 except for the time delay of 5.0 (time units) that is added to the system.

The difference between having the time delay and not is a theoretical process control system and an actual process control system.

The graphs corresponding to problem 2: part a in figures 16-17 are very similar to the graphs that were discussed in part 7 and 8. The only difference (again) is the flat time delay for 5 time units before the system starts showing any change.

For part a; regulatory control code the integral gains were set to $K_{ci1} = K_{c1}/10.0$ and $K_{ci2} = K_{c2}/22.0$

For part a; servo control code the integral gains were set to $K_{ci1} = K_{c1}/27.0$ and $K_{ci2} = K_{c2}/27.0$

In both codes of part a, the code was changed to accommodate time delay. The code can be found in appendix I.

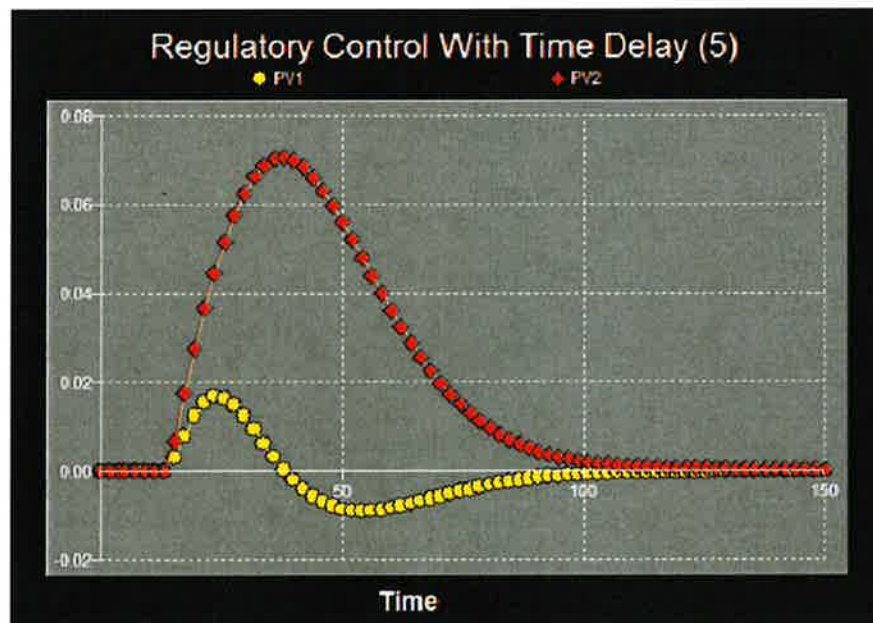


Figure 16: Problem 2: Part 10; Regulatory Control With Time Delay

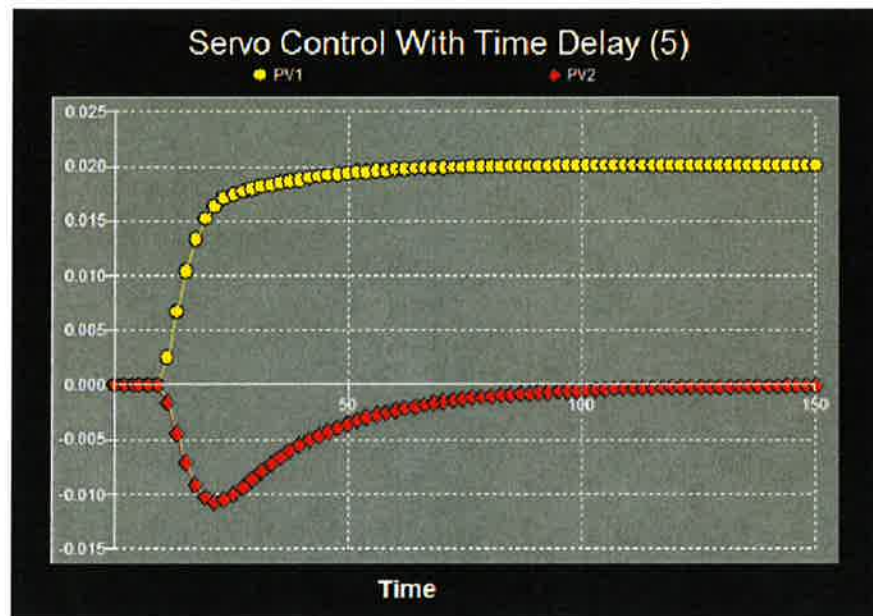


Figure 17: Problem 2: Part 10; Servo Control With Time Delay

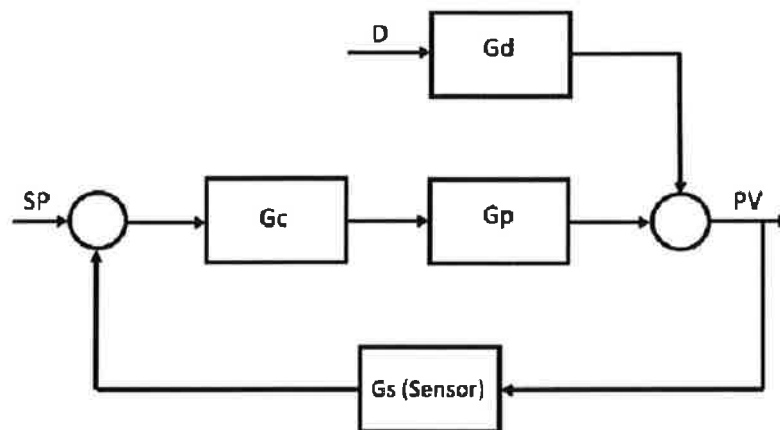


Figure 18: Process diagram of disturbance after process

$$PV = DG_D + G_C G_P (SP - PV G_S) \quad (135)$$

When the set point is set to zero, the equation becomes

$$PV = DG_D - PVG_C G_P G_S \quad (136)$$

$$PV(1 + G_C G_P G_S) = DG_D \quad (137)$$

$$PV = \frac{DG_D}{1 + G_C G_P G_S} \quad (138)$$

the closed loop transfer function becomes the denominator

$$0 = 1 + G_C G_P G_S \quad (139)$$

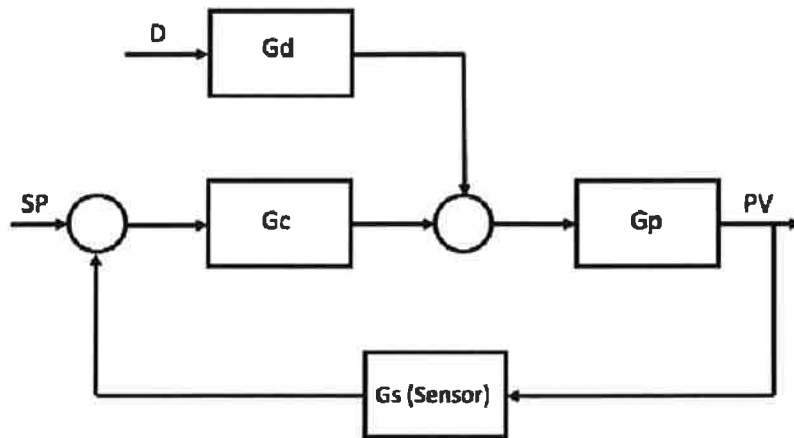


Figure 19: Process Diagram of disturbance before process

$$PV = G_P(DG_D + G_C(SP - PVG_S)) \quad (140)$$

When the set point is set to zero, the equation becomes

$$PV = DG_D G_P + PVG_C G_P G_S \quad (141)$$

$$PV(1 + G_C G_P G_S) = DG_D G_P \quad (142)$$

$$PV = \frac{DG_D G_P}{1 + G_C G_P G_S} \quad (143)$$

the closed loop transfer function becomes the denominator

$$0 = 1 + G_C G_P G_S \quad (144)$$

in both cases, the disturbance coming after the process and the disturbance coming after the control, the proof

$$G_{Pij} = G_{Pij} * e^{s(\text{AnalyzerDeadTime})} \quad (145)$$

is proven to be true because the closed loop transfer function is the same for both

For problem 2: part c; tuning methods, it was required to first find the ultimate period and the ultimate gain. The process used to find the two values is shown below

For number 10 it is given that

$$K_{C1} = 32.08 \quad (146)$$

$$K_{C2} = 3.58 \quad (147)$$

To plug in the tuning methods, the ultimate gain had to be solved. This was done using Bode's Method for Ultimate Gain. The equations used for K_{C1} are as follows

$$\tau_{PI1}\omega_1 + \tan(\theta_P\omega_1) = 0 \quad (148)$$

Because it was given in the problem statement that

$$\tau_{I1} = 7.0 \quad (149)$$

$$\theta_P = 5.0 \quad (150)$$

$$7.0\omega_1 + \tan(5.0\omega_1) = 0 \quad (151)$$

Using goal seek it was discovered that

$$\omega_1 = 0.367 \quad (152)$$

ω_1 is plugged into the K_C equation to solve for the ultimate gain

$$K_{U1} = -\frac{1}{0.1245\cos(5.0 * 0.367)} \quad (153)$$

the Ultimate Gain was calculated to be

$$K_{U1} = 30.58 \quad (154)$$

The same process is performed for K_{U2} and calculated to be

$$K_{U2} = 5.091 \quad (155)$$

The Ultimate Period P_U was determined for both ultimate gains using the ω that was found using goal seek. The formulas used are shown below

$$P_{U1} = \frac{2\pi}{\omega_1} \quad (156)$$

$$P_{U1} = \frac{2\pi}{0.367} \quad (157)$$

$$P_{U1} = 17.11 \quad (158)$$

$$P_{U2} = \frac{2\pi}{\omega_2} \quad (159)$$

$$P_{U2} = \frac{2\pi}{0.3495} \quad (160)$$

$$P_{U2} = 17.975 \quad (161)$$

- The Ziegler Nichols Method was developed for self regulating processes described by the process function shown below

$$G_P(s) = \frac{K_P e^{s t_d}}{\tau s + 1} \quad (162)$$

This specific method uses quarter-amplitude damping which causes the process to overshoot its set-point and to oscillate a few times before it settles down. To prevent excessive overshooting and to increase robustness of the Z-N method, the tuning rules are modified by adding a stability margin (SM). The SM values are found to be between 1 and 4.

This method may apply to interactive controllers if the controller has a different algorithm conversion shown below

$$PB = \frac{100\%}{K_C} \quad (163)$$

For a proportional-integral controller the K_C can be found by the following equation

$$K_C = \left(\frac{0.9}{SM(K_P)} \right) \left(\frac{\tau}{t_d} \right) \quad (164)$$

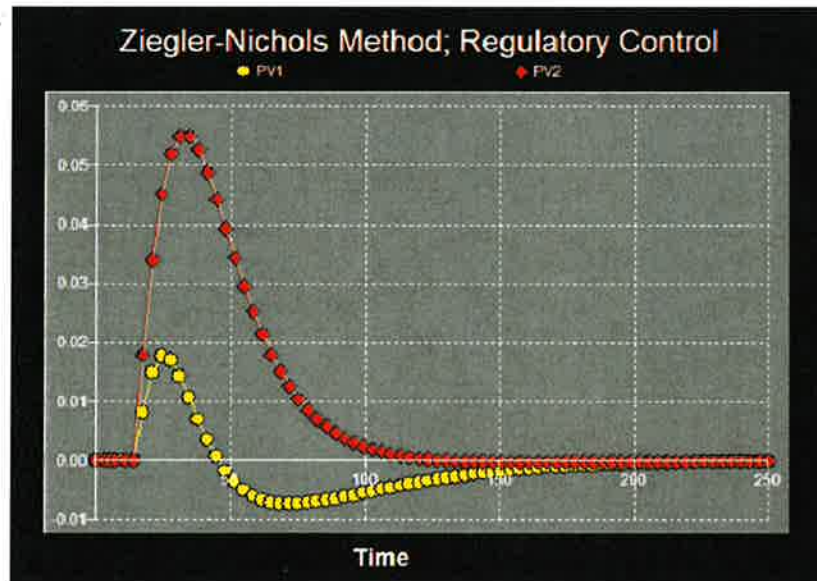


Figure 20: Problem 2: Part 10; Ziegler-Nichols Method:Regulatory Control With Time Delay

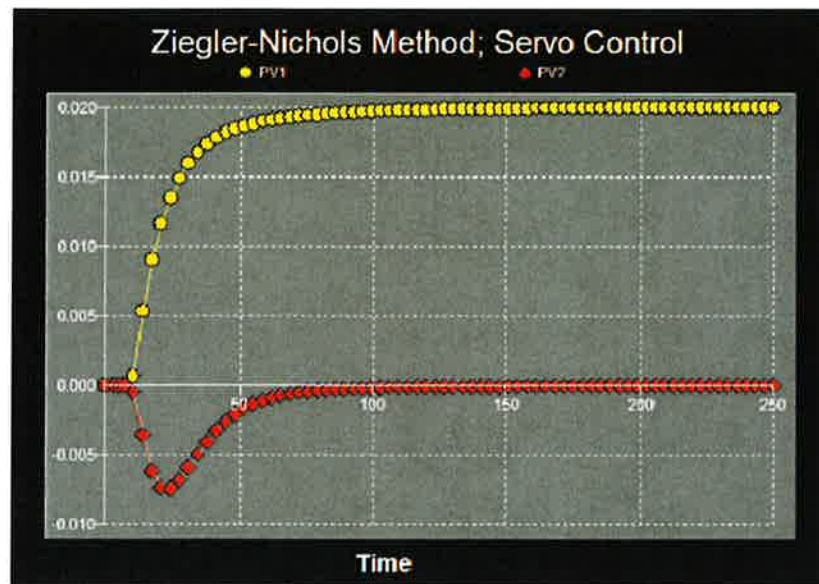


Figure 21: Problem 2: Part 10; Ziegler-Nichols Method: Servo Control With Time Delay

- The Ziegler-Nichols Ultimate Gain Method is used as a "last-resort" method and is to be done from experienced operators. The idea of this method is to bring the proportional only loop control into a constant amplitude oscillation by changing the controller gain up or down. The value of the controller gain that is accomplished is called the "ultimate gain K_U " and the period between the constant amplitude oscillations is called the "ultimate period P_U "

For a proportional-integral controller the K_C can be found by the following equation

$$K_C = \frac{0.45K_U}{SM} \quad (165)$$

$$\tau_I = 0.83P_U \quad (166)$$

Student ID: 667780356

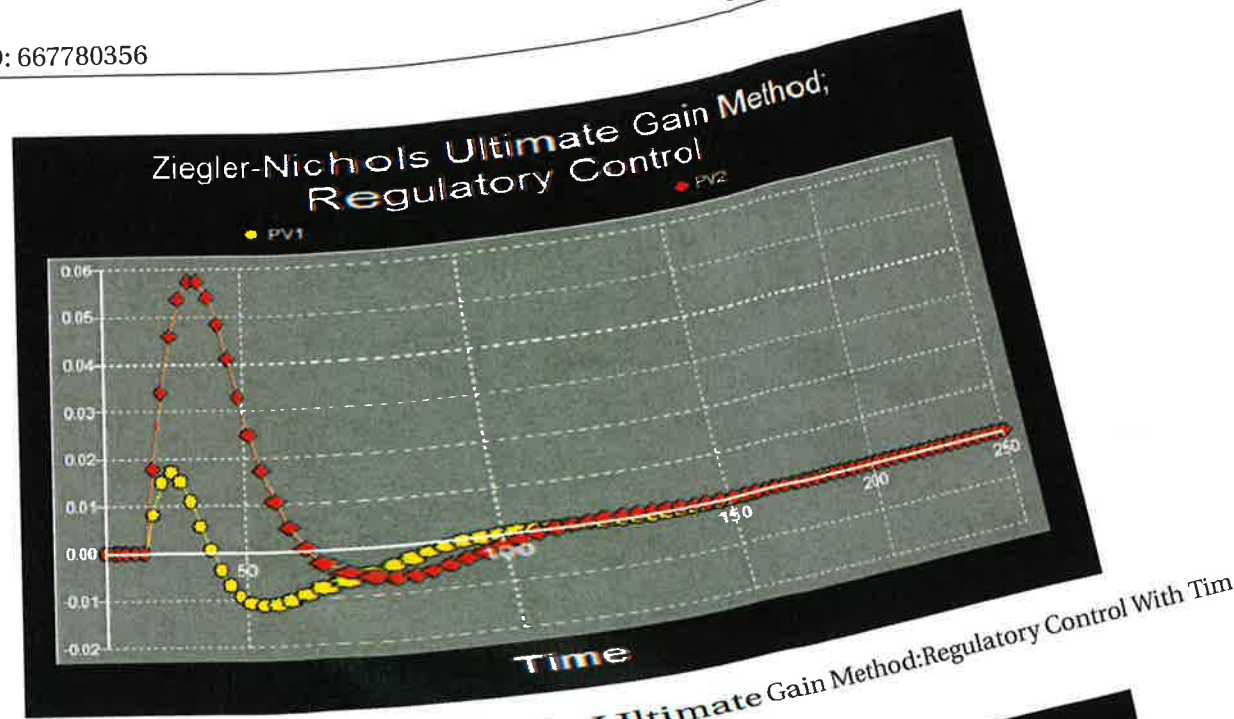


Figure 22: Problem 2: Part 10; Ziegler-Nichols Ultimate Gain Method; Regulatory Control With Time Delay

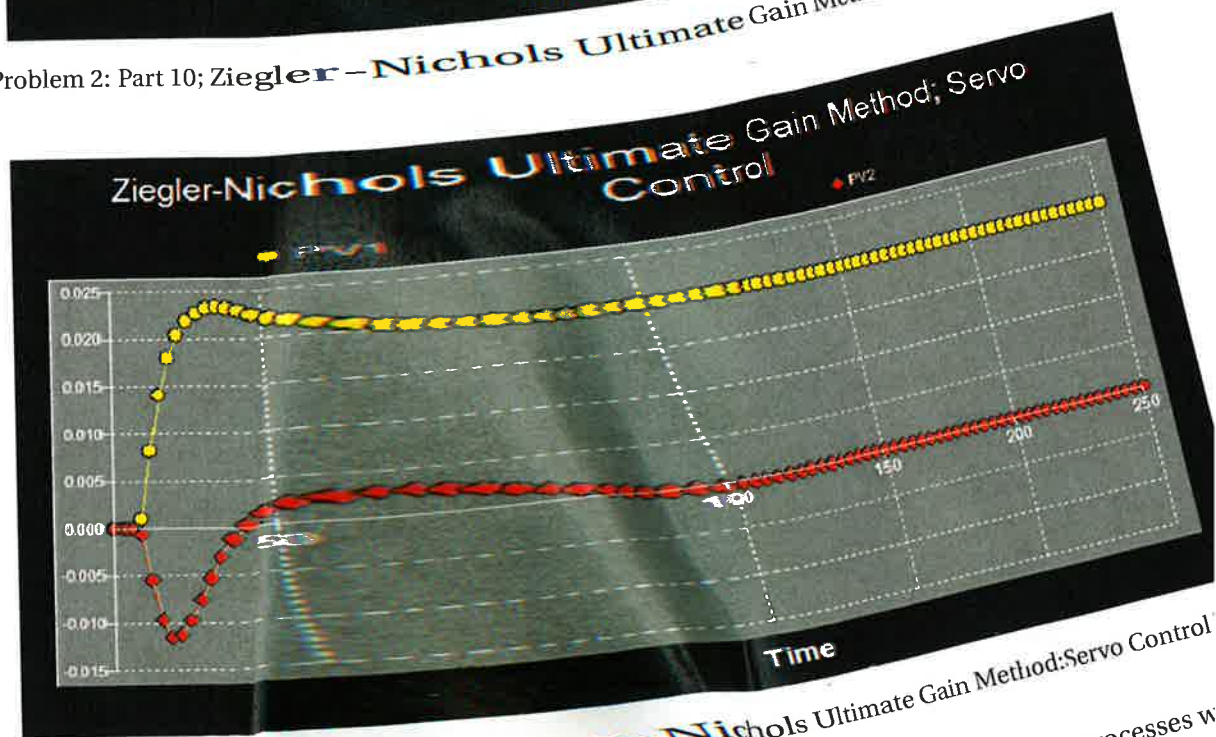


Figure 23: Problem 2: Part 10; Ziegler-Nichols Ultimate Gain Method; Servo Control

- Cohen-Coon Method works really well on lag-dominant processes with

$$\tau_d \leq 4\tau$$

The CC method aims for quarter-amplitude damping and they were developed for a NON-INTERACTIVE algorithm

For a PD Controllers the K_C can be found by the following equation

$$K_C = \left(\frac{0.9}{(SM)K_P} \right) \left(\frac{\tau}{t_d} + 0.092 \right) \quad (168)$$

$$\tau_I = 3.33 t_d \left(\frac{\tau + 0.092 t_d}{\tau + 2.2 t_d} \right) \quad (169)$$

For a PI Controllers the K_C can be found by the following equation

$$K_C = \left(\frac{1.24}{(SM)K_P} \right) \left(\frac{\tau}{t_d} + 0.129 \right) \quad (170)$$

$$\tau_D = 0.27 t_d \left(\frac{\tau + 0.185 t_d}{\tau + 0.611 t_d} \right) \quad (171)$$

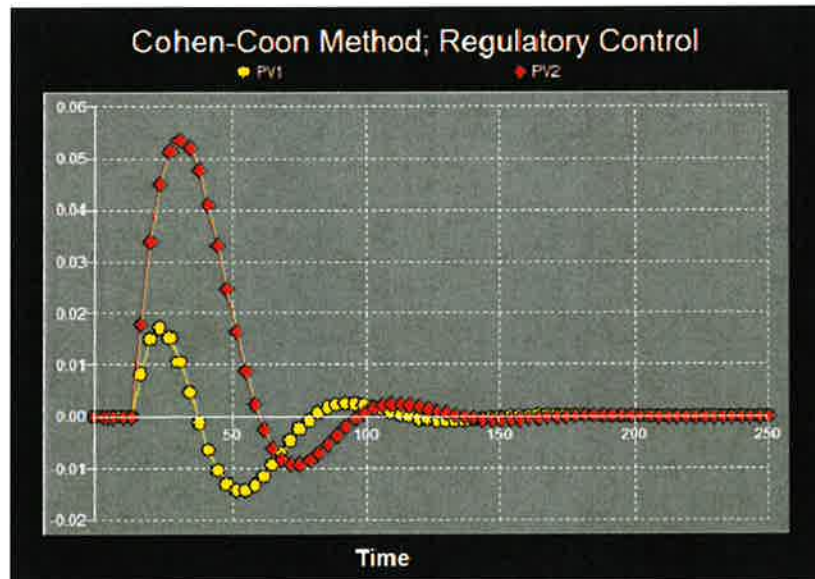


Figure 24: Problem 2: Part 10; Cohen-Coon Method:Regulatory Control With Time Delay

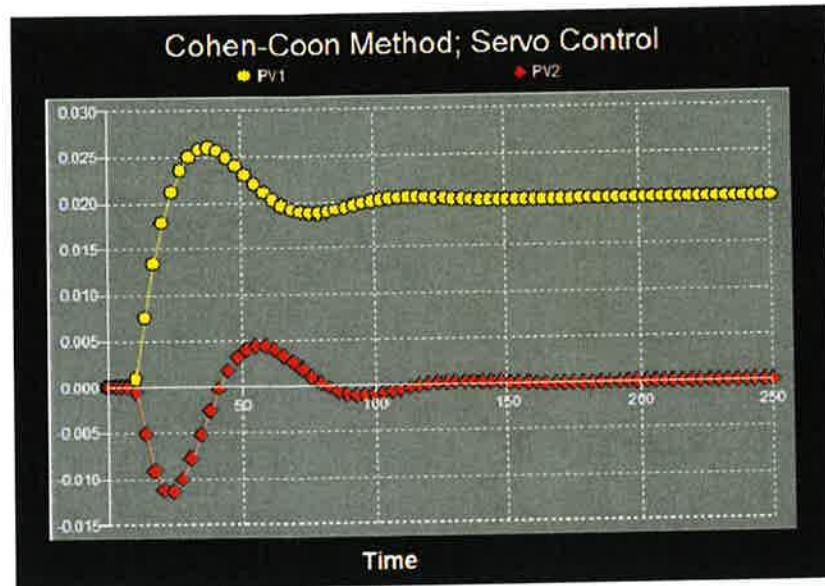


Figure 25: Problem 2: Part 10; Cohen-Coon Method: Servo Control With Time Delay

The decisions of which tuning method is best for which control is based on which graph shows the least amount of oscillation and which graph achieved the desired set point or settles the disturbance the fastest.

For Servo control the Ziegler-Nichols method was the best

For Regulatory control the Ziegler-Nichols ultimate gain method appeared to have the least amount of oscillations and therefore must have been the best at tuning.

Reference

- Professor Caracotsios' Lecture Notes

```

!PID Controllers
!*****
!Regulatory-Control Implementation in Time-Domain
!*****
!Written by: Professor Michael Caracotsios and Kelsey Jensen
!*****

Use AthenaUtilities
Global Tau11,Tau12,Tau21,Tau22,TauD1,TauD2 As Real
Global ThetaD1,ThetaD2,Kp11,Kp12,Kp21,Kp22 As Real
Global KpD1,KpD2,Taud,Thetap,Kd,Ku,Pu As Real
Global Kc1,Kc2,Ti,Kcli,Kc2i,Td,Kcd,SP1,SP2,DS As Real
Global Time(5000),Y1i(5000),Y2i(5000) As Real
Global Y3i(5000),Y4i(5000),Y5i(5000),Y6i(5000) As Real
Global CO1i(5000),CO2i(5000),PV1i(5000),PV2i(5000) As Real
Global Eli(5000),E2i(5000) As Real
Global iCount As Integer

!*****
!Process Gain for process as well as disturbance; ELJ
Kp11=0.1247 !ELJ
Kp12=0.546 !ELJ
Kp21=-0.1246 !ELJ
Kp22=1.116 !ELJ
KpD1=0.588 !ELJ
KpD2=1.41 !ELJ

!*****
!Process Lag for process as well as disturbance; ELJ
Tau11=10.0 !ELJ
Tau12=24.0 !ELJ
Tau21=16.0 !ELJ
Tau22=22.0 !ELJ
TauD1=22.0 !ELJ
TauD2=24.0 !ELJ

!*****
!Time Delay for both process and disturbance; ELJ
ThetaD1=7.0 !ELJ
ThetaD2=7.0 !ELJ
thetap=0.0 !ELJ

!*****
!Controller Gains for both processes; ELJ
Kc1=32.08 !Proportional Gain; ELJ
Kc2=3.58 !Proportional Gain; ELJ
Kcli=32.08/10.0 !Integral Gain; ELJ
Kc2i=3.58/22.0 !Integral Gain; ELJ
Kcd=0.0 !Derivative Gain; ELJ

!*****
iCount=0 !Counter for the History Profiles; ELJ
SP1=0.0 !Process Variable Set Point 1 Value; ELJ
SP2=0.0 !Process Variable Set Point 2 Value; ELJ
DS=0.1 !Disturbance Set Point Value; ELJ

!*****
@Initial Conditions
U(1)=0.0 !ELJ
U(2)=0.0 !ELJ
U(3)=0.0 !ELJ
U(4)=0.0 !ELJ
U(5)=0.0 !ELJ
U(6)=0.0 !ELJ

!*****
@Model Equations

```

```

Dim PV1,PV2,CO1,CO2,IE As Real          !ELJ
Dim i As Integer                        !ELJ

!*****
!Collect the History of the Integrated Model Variables

If (iRes==9)Then                        !ELJ
    iCount=iCount+1.0                  !ELJ
    Time(iCount)=T                     !ELJ
    Y1i(iCount)=U(1)                   !ELJ
    Y2i(iCount)=U(2)                   !ELJ
    Y3i(iCount)=U(3)                   !ELJ
    Y4i(iCount)=U(4)                   !ELJ
    Y5i(iCount)=U(5)                   !ELJ
    Y6i(iCount)=U(6)                   !ELJ
EndIf                                   !ELJ

!*****
!Formulate the Feedback Control Loop Equations

Select Case(iRange)                    !ELJ

    Case(0)                             !ELJ
        PV1=U(1)+U(2)                  !ELJ
        If (iRes==9) E1i(iCount)=SP1-PV1 !ELJ
        CO1=Kc1*(SP1-PV1)+Kcli*(SP1-PV1)*T !ELJ
        PV2=U(3)+U(4)                  !ELJ
        If (iRes==9) E2i(iCount)=SP2-PV2 !ELJ
        CO2=Kc2*(SP2-PV2)+Kc2i*(SP2-PV2)*T !ELJ

        F(1)=-U(1)+Kp11*CO1            !ELJ
        F(2)=-U(2)+Kp12*CO2            !ELJ
        F(3)=-U(3)+Kp21*CO1            !ELJ
        F(4)=-U(4)+Kp22*CO2            !ELJ
        F(5)=-U(5)                     !ELJ
        F(6)=-U(6)                     !ELJ

    Case(1)                             !ELJ
        PV1=U(1)+U(2)+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-ThetaD1)
        If (iRes==9) E1i(iCount)=SP1-PV1 !ELJ
        IE=0.0                          !ELJ
        Do i=1,iCount-1                 !ELJ
            IE=IE+(E1i(i+1)+E1i(i))*(Time(i+1)-Time(i))/2.0 !ELJ
        EndDo                           !ELJ
        IE=IE+(E1i(iCount)+SP1-PV1)*(T-Time(iCount))/2.0 !ELJ
        CO1=Kc1*(SP1-PV1)+Kcli*IE        !ELJ

        PV2=U(3)+U(4)+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-ThetaD2)
        If (iRes==9) E2i(iCount)=SP2-PV2 !ELJ
        IE=0.0                          !ELJ
        Do i=1,iCount-1                 !ELJ
            IE=IE+(E2i(i+1)+E2i(i))*(Time(i+1)-Time(i))/2.0 !ELJ
        EndDo                           !ELJ
        IE=IE+(E2i(iCount)+SP2-PV2)*(T-Time(iCount))/2.0 !ELJ
        CO2=Kc2*(SP2-PV2)+Kc2i*IE        !ELJ

        F(1)=-U(1)+Kp11*CO1            !ELJ
        F(2)=-U(2)+Kp12*CO2            !ELJ
        F(3)=-U(3)+Kp21*CO1            !ELJ
        F(4)=-U(4)+Kp22*CO2            !ELJ
        F(5)=-U(5)+KpD1*DS              !ELJ
        F(6)=-U(6)+KpD2*DS              !ELJ
End Select                             !ELJ

!*****
!Collect History of the Process Variables and Control Output

```

```

If (iRes==9)Then                                !ELJ
  PV1i(iCount)=PV1                                !ELJ
  PV2i(iCount)=PV2                                !ELJ
  CO1i(iCount)=CO1                                !ELJ
  CO2i(iCount)=CO2                                !ELJ
EndIf

! *****
@Coefficient Matrix
E(1)=Tau11                                !ELJ
E(2)=Tau12                                !ELJ
E(3)=Tau21                                !ELJ
E(4)=Tau22                                !ELJ
E(5)=TauD1                                !ELJ
E(6)=TauD2                                !ELJ

! *****
@Solver Options
Neq=6                                !ELJ
Npts=75.0                                !ELJ
Tend=250.0                                !ELJ
Tstop=7                                !ELJ

Param(28)=1                                !Continue Integration Past Dead Time; ELJ
Param(177)=1                                !Enable Integration History Access; ELJ
VariableNames=Time;Y1;Y2;Y3;Y4;Y5;Y6      !Variable Names; ELJ
Append=PV1;PV2                                !Display Process Variable; ELJ

! *****
@After Calling Solver
Dim PV1,PV2,CO1,CO2 As Real                                !ELJ
If(iCount==0)Then                                !ELJ
  PV1=U(1)+U(2)+U(5)                                !ELJ
  PV2=U(3)+U(4)+U(6)                                !ELJ
  CO1=Kc1*(SP1-PV1)                                !ELJ
  CO2=Kc2*(SP2-PV2)                                !ELJ
Else                                !ELJ
  PV1=QuadraticSpline(Time(1:iCount),PV1i(1:iCount),Tini)!ELJ
  PV2=QuadraticSpline(Time(1:iCount),PV2i(1:iCount),Tini)!ELJ
  CO1=QuadraticSpline(Time(1:iCount),CO1i(1:iCount),Tini)!ELJ
  CO2=QuadraticSpline(Time(1:iCount),CO2i(1:iCount),Tini)!ELJ
EndIf                                !ELJ

```

```

!PI Controllers
!*****
!Servo-Control Implementation in Time-Domain
!*****
!Written by: Professor Michael Caracotsios and Kelsey Jensen
!*****

Use AthenaUtilities
Global Tau11,Tau12,Tau21,Tau22,TauD1,TauD2 As Real
Global ThetaD1,ThetaD2,Kp11,Kp12,Kp21,Kp22 As Real
Global KpD1,KpD2,Taud,Thetap,Kd,Ku,Pu As Real
Global Kc1,Kc2,Ti,Kcli,Kc2i,Td,Kcd,SP1,SP2,DS As Real
Global Time(5000),Y1i(5000),Y2i(5000) As Real
Global Y3i(5000),Y4i(5000),Y5i(5000),Y6i(5000) As Real
Global CO1i(5000),CO2i(5000),PV1i(5000),PV2i(5000) As Real
Global Eli(5000),E2i(5000) As Real
Global iCount As Integer

!*****
!Process Gain for process as well as disturbance; ELJ
Kp11=0.1247                                !ELJ
Kp12=0.546                                !ELJ
Kp21=-0.1246                               !ELJ
Kp22=1.116                                !ELJ
KpD1=0.588                                !ELJ
KpD2=1.41                                 !ELJ

!*****
!Process Lag for process as well as disturbance; ELJ
Tau11=10.0                                !ELJ
Tau12=24.0                                !ELJ
Tau21=16.0                                !ELJ
Tau22=22.0                                !ELJ
TauD1=22.0                                !ELJ
TauD2=24.0                                !ELJ

!*****
!Time Delay for both process and disturbance; ELJ
ThetaD1=7.0                                !ELJ
ThetaD2=7.0                                !ELJ
thetap=0.0                                !ELJ

!*****
!Controller Gains for both processes; ELJ
Kc1=32.08                                !Proportional Gain; ELJ
Kc2=3.58                                 !Proportional Gain; ELJ
Kcli=32.08/10.0                           !Integral Gain; ELJ
Kc2i=3.58/22.0                             !Integral Gain; ELJ
Kcd=0.0                                  !Derivative Gain; ELJ

!*****
iCount=0                                !Counter for the History Profiles; ELJ
SP1=0.02                                !Process Variable Set Point 1 Value; ELJ
SP2=0.0                                !Process Variable Set Point 2 Value; ELJ
DS=0.0                                !Disturbance Set Point Value; ELJ

!*****
@Initial Conditions
U(1)=0.0                                !ELJ
U(2)=0.0                                !ELJ
U(3)=0.0                                !ELJ
U(4)=0.0                                !ELJ
U(5)=0.0                                !ELJ
U(6)=0.0                                !ELJ

!*****
@Model Equations

```

```

Dim PV1,PV2,CO1,CO2,IE As Real          !ELJ
Dim i As Integer                        !ELJ

!*****
!Collect the History of the Integrated Model Variables

If (iRes==9)Then                        !ELJ
    iCount=iCount+1.0                  !ELJ
    Time(iCount)=T                     !ELJ
    Y1i(iCount)=U(1)                   !ELJ
    Y2i(iCount)=U(2)                   !ELJ
    Y3i(iCount)=U(3)                   !ELJ
    Y4i(iCount)=U(4)                   !ELJ
    Y5i(iCount)=U(5)                   !ELJ
    Y6i(iCount)=U(6)                   !ELJ
EndIf                                   !ELJ

!*****
!Formulate the Feedback Control Loop Equations

Select Case(iRange)                   !ELJ

    Case(0)                             !ELJ
        PV1=U(1)+U(2)                  !ELJ
        If(iRes==9)E1i(iCount)=SP1-PV1 !ELJ
        CO1=Kc1*(SP1-PV1)+Kcli*(SP1-PV1)*T !ELJ
        PV2=U(3)+U(4)                  !ELJ
        If(iRes==9)E2i(iCount)=SP2-PV2 !ELJ
        CO2=Kc2*(SP2-PV2)+Kc2i*(SP2-PV2)*T !ELJ

        F(1)=-U(1)+Kp11*CO1             !ELJ
        F(2)=-U(2)+Kp12*CO2             !ELJ
        F(3)=-U(3)+Kp21*CO1             !ELJ
        F(4)=-U(4)+Kp22*CO2             !ELJ
        F(5)=-U(5)                     !ELJ
        F(6)=-U(6)                     !ELJ

    Case(1)                             !ELJ
        PV1=U(1)+U(2)+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-ThetaD1)
        If(iRes==9)E1i(iCount)=SP1-PV1 !ELJ
        IE=0.0                          !ELJ
        Do i=1,iCount-1                 !ELJ
            IE=IE+(E1i(i+1)+E1i(i))*(Time(i+1)-Time(i))/2.0 !ELJ
        EndDo                           !ELJ
        IE=IE+(E1i(iCount)+SP1-PV1)*(T-Time(iCount))/2.0 !ELJ
        CO1=Kc1*(SP1-PV1)+Kcli*IE        !ELJ

        PV2=U(3)+U(4)+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-ThetaD2)
        If(iRes==9)E2i(iCount)=SP2-PV2 !ELJ
        IE=0.0                          !ELJ
        Do i=1,iCount-1                 !ELJ
            IE=IE+(E2i(i+1)+E2i(i))*(Time(i+1)-Time(i))/2.0 !ELJ
        EndDo                           !ELJ
        IE=IE+(E2i(iCount)+SP2-PV2)*(T-Time(iCount))/2.0 !ELJ
        CO2=Kc2*(SP2-PV2)+Kc2i*IE        !ELJ

        F(1)=-U(1)+Kp11*CO1             !ELJ
        F(2)=-U(2)+Kp12*CO2             !ELJ
        F(3)=-U(3)+Kp21*CO1             !ELJ
        F(4)=-U(4)+Kp22*CO2             !ELJ
        F(5)=-U(5)+KpD1*DS               !ELJ
        F(6)=-U(6)+KpD2*DS               !ELJ
End Select                             !ELJ

!*****
!Collect History of the Process Variables and Control Output

```

```

If (iRes==9)Then                                !ELJ
  PV1i(iCount)=PV1                                !ELJ
  PV2i(iCount)=PV2                                !ELJ
  CO1i(iCount)=CO1                                !ELJ
  CO2i(iCount)=CO2                                !ELJ
EndIf

!*****
@Coefficient Matrix
E(1)=Tau11                                !ELJ
E(2)=Tau12                                !ELJ
E(3)=Tau21                                !ELJ
E(4)=Tau22                                !ELJ
E(5)=TauD1                                !ELJ
E(6)=TauD2                                !ELJ

!*****
@Solver Options
Neg=6                                !ELJ
Npts=75.0                                !ELJ
Tend=250.0                                !ELJ
Tstop=7                                !ELJ

Param(28)=1                                !Continue Integration Past Dead Time; ELJ
Param(177)=1                                !Enable Integration History Access; ELJ
VariableNames=Time;Y1;Y2;Y3;Y4;Y5;Y6      !Variable Names; ELJ
Append=PV1;PV2                                !Display Process Variable; ELJ

!*****
@After Calling Solver
Dim PV1,PV2,CO1,CO2 As Real                                !ELJ
If(iCount==0)Then                                !ELJ
  PV1=U(1)+U(2)+U(5)                                !ELJ
  PV2=U(3)+U(4)+U(6)                                !ELJ
  CO1=Kc1*(SP1-PV1)                                !ELJ
  CO2=Kc2*(SP2-PV2)                                !ELJ
Else                                !ELJ
  PV1=QuadraticSpline(Time(1:iCount),PV1i(1:iCount),Tini)!ELJ
  PV2=QuadraticSpline(Time(1:iCount),PV2i(1:iCount),Tini)!ELJ
  CO1=QuadraticSpline(Time(1:iCount),CO1i(1:iCount),Tini)!ELJ
  CO2=QuadraticSpline(Time(1:iCount),CO2i(1:iCount),Tini)!ELJ
EndIf                                !ELJ

```



```

!PID Controllers
!*****
!Regulatory + Servo Control Implementation in Time-Domain
!*****
!Written by: Professor Michael Caracotsios and Elizabeth Jensen
!*****

Use AthenaUtilities
Global Tau11,Tau12,Tau21,Tau22,TauD1,TauD2,ThetaD1,ThetaD2,Thetap,Kp11 As Real
Global Kp12,Kp21,Kp22,KpD1,KpD2,Taud,Thetad,Kd,Ku1,Ku2,Pu1,Pu2 As Real
Global Kc1,Kc2,Ti,Kcli,Kc2i,Td,Kcd,SP1,SP2,DS, SM, r As Real
Global Time(5000),Y1i(5000),Y2i(5000),Y3i(5000),Y4i(5000),Y5i(5000),Y6i(5000) As Real
Global CO1i(5000),CO2i(5000),PV1i(5000),PV2i(5000),E1i(5000),E2i(5000) As Real
Global iCount As Integer
r=8 !8 = regulatory control; 9 = servo control - ELJ

!*****
!Process Gain for process as well as disturbance; ELJ
Kp11=0.1247 !ELJ
Kp12=0.546 !ELJ
Kp21=-0.1246 !ELJ
Kp22=1.116 !ELJ
KpD1=0.588 !ELJ
KpD2=1.41 !ELJ

!*****
!Process Lag for process as well as disturbance; ELJ
Tau11=10.0 !ELJ
Tau12=24.0 !ELJ
Tau21=16.0 !ELJ
Tau22=22.0 !ELJ
TauD1=22.0 !ELJ
TauD2=24.0 !ELJ

!*****
!Time Delay for both process and disturbance; ELJ
ThetaD1=7.0 !ELJ
ThetaD2=7.0 !ELJ
Thetap=5.0 !ELJ

!*****
If(r==8)Then !Controler Gains used for regulatory control; ELJ
SM=4.0 !Stability Margin; ELJ
Kc1=32.08/SM !Proportional Gain for control 1; ELJ
Kc2=3.58/SM !Proportional Gain for control 2; ELJ
Kcli=Kc1/10 !Integral Gain for control 1; ELJ
Kc2i=Kc2/22 !Integral Gain for control 2; ELJ
Kcd=0.0 !Derivative Gain, No derivative action present; ELJ
EndIf

!*****
If(r==9)Then !Controler Gains used for Servo control; ELJ
SM=4.0 !Stability Margin; ELJ
Kc1=32.08/SM !Proportional Gain for control 1; ELJ
Kc2=3.58/SM !Proportional Gain for control 2; ELJ
Kcli=Kc1/27.0 !Integral Gain for control 1; ELJ
Kc2i=Kc2/27.0 !Integral Gain for control 2; ELJ
Kcd=0.0 !Derivative Gain, No derivative action present; ELJ
EndIf

!*****
If(r==8)Then !Values used for Regulatory Control; ELJ
iCount=0 !Counter for the History Profiles; ELJ
SP1=0.0 !Process Variable Set Point 1 Value; ELJ
SP2=0.0 !Process Variable Set Point 2 Value; ELJ
DS=0.1 !Disturbance Set Point Value; ELJ
EndIf

```

```

! *****
If (r==9) Then          !Values used for Regulatory Control; ELJ
  iCount=0              !Counter for the History Profiles; ELJ
  SP1=0.02              !Process Variable Set Point 1 Value; ELJ
  SP2=0.0               !Process Variable Set Point 2 Value; ELJ
  DS=0.0                !Disturbance Set Point Value; ELJ
EndIf

! *****
@Initial Conditions
  U(1)=0.0              !ELJ
  U(2)=0.0              !ELJ
  U(3)=0.0              !ELJ
  U(4)=0.0              !ELJ
  U(5)=0.0              !ELJ
  U(6)=0.0              !ELJ

! *****
@Model Equations
  Dim PV1,PV2,CO1,CO2,IE As Real      !ELJ
  Dim i As Integer                    !ELJ

! *****
!Collect the History of the Integrated Model Variables
If (iRes==9) Then
  iCount=iCount+1.0                !ELJ
  Time(iCount)=T                   !ELJ
  Y1i(iCount)=U(1)                 !ELJ
  Y2i(iCount)=U(2)                 !ELJ
  Y3i(iCount)=U(3)                 !ELJ
  Y4i(iCount)=U(4)                 !ELJ
  Y5i(iCount)=U(5)                 !ELJ
  Y6i(iCount)=U(6)                 !ELJ
EndIf

! *****
!Formulate the Feedback Control Loop Equations

Select Case (iRange)               !ELJ

  Case (0)                          !ELJ
    PV1=U(1)+U(2)                   !ELJ
    If (iRes==9) E1i(iCount)=SP1-PV1 !ELJ
      CO1=Kc1*(SP1-PV1)+Kcli*(SP1-PV1)*T !ELJ

    PV2=U(3)+U(4)                   !ELJ
    If (iRes==9) E2i(iCount)=SP2-PV2 !ELJ
      CO2=Kc2*(SP2-PV2)+Kc2i*(SP2-PV2)*T !ELJ

    F(1)=0.0                        !ELJ
    F(2)=0.0                        !ELJ
    F(3)=0.0                        !ELJ
    F(4)=0.0                        !ELJ
    F(5)=0.0                        !ELJ
    F(6)=0.0                        !ELJ

  Case (1)

    PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-
5)+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-5)
    If (iRes==9) E1i(iCount)=SP1-PV1 !ELJ
    IE=0.0                          !ELJ
    Do i=1,iCount-1                 !ELJ
      IE=IE+(E1i(i+1)+E1i(i))*(Time(i+1)-Time(i))/2.0 !ELJ

```

```

EndDo
IE=IE+(E1i(iCount)+SP1-PV1)*(T-Time(iCount))/2.0 !ELJ
CO1=Kc1*(SP1-PV1)+Kcli*IE !ELJ
PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-
5)+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-5)

If(iRes==9)E2i(iCount)=SP2-PV2 !ELJ
IE=0.0 !ELJ
Do i=1,iCount-1 !ELJ
  IE=IE+(E2i(i+1)+E2i(i))*(Time(i+1)-Time(i))/2.0 !ELJ
EndDo !ELJ
IE=IE+(E2i(iCount)+SP2-PV2)*(T-Time(iCount))/2.0 !ELJ
CO2=Kc2*(SP2-PV2)+Kc2i*IE !ELJ

F(1)=-U(1)+Kp11*CO1 !ELJ

F(2)=-U(2)+Kp12*CO2 !ELJ
F(3)=-U(3)+Kp21*CO1 !ELJ
F(4)=-U(4)+Kp22*CO2 !ELJ
F(5)=0.0 !ELJ

F(6)=0.0 !ELJ

Case(2) !ELJ
PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-
5)+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-
5)+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-7) !ELJ
If(iRes==9)E1i(iCount)=SP1-PV1 !ELJ
IE=0.0 !ELJ
Do i=1,iCount-1 !ELJ
  IE=IE+(E1i(i+1)+E1i(i))*(Time(i+1)-Time(i))/2.0 !ELJ
EndDo !ELJ
IE=IE+(E1i(iCount)+SP1-PV1)*(T-Time(iCount))/2.0 !ELJ
CO1=Kc1*(SP1-PV1)+Kcli*IE !ELJ
PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-
5)+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-
5)+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-7) !ELJ

If(iRes==9)E2i(iCount)=SP2-PV2 !ELJ
IE=0.0 !ELJ
Do i=1,iCount-1 !ELJ
  IE=IE+(E2i(i+1)+E2i(i))*(Time(i+1)-Time(i))/2.0 !ELJ
EndDo !ELJ
IE=IE+(E2i(iCount)+SP2-PV2)*(T-Time(iCount))/2.0 !ELJ
CO2=Kc2*(SP2-PV2)+Kc2i*IE !ELJ

F(1)=-U(1)+Kp11*CO1 !ELJ

F(2)=-U(2)+Kp12*CO2 !ELJ
F(3)=-U(3)+Kp21*CO1 !ELJ
F(4)=-U(4)+Kp22*CO2 !ELJ
F(5)=-U(5)+KpD1*DS !ELJ

F(6)=-U(6)+KpD2*DS !ELJ

End Select !ELJ

!*****
!Collect History of the Process Variables and Control Output

```

```

If (iRes==9)Then                                !ELJ
  PV1i(iCount)=PV1                                !ELJ
  PV2i(iCount)=PV2                                !ELJ
  CO1i(iCount)=CO1                                !ELJ
  CO2i(iCount)=CO2                                !ELJ
EndIf                                              !ELJ

! *****
@Coefficient Matrix
E(1)=Tau11                                !ELJ
E(2)=Tau12                                !ELJ
E(3)=Tau21                                !ELJ
E(4)=Tau22                                !ELJ
E(5)=TauD1                                !ELJ
E(6)=TauD2                                !ELJ

! *****
@Solver Options
Neq=6                                !ELJ
Npts=75.0                                !ELJ
Tend=150.0                                !ELJ
Tstop=5.0;7.0                                !ELJ

Param(28)=1      !Continue Integration Past Dead Time; ELJ
Param(177)=1     !Enable Integration History Access; ELJ
VariableNames=Time;Y1;Y2;Y3;Y4;Y5;Y6 !Variable Names; ELJ
Append=PV1;PV2      !Display Process Variable; ELJ

! *****
@After Calling Solver
Dim PV1,PV2,CO1,CO2 As Real                                !ELJ
If(iCount==0)Then                                !ELJ
  PV1=U(1)+U(2)+U(5)                                !ELJ
  PV2=U(3)+U(4)+U(6)                                !ELJ
  CO1=Kc1*(SP1-PV1)                                !ELJ
  CO2=Kc2*(SP2-PV2)                                !ELJ
Else                                              !ELJ
  PV1=QuadraticSpline(Time(1:iCount),PV1i(1:iCount),Tini)
  PV2=QuadraticSpline(Time(1:iCount),PV2i(1:iCount),Tini)
  CO1=QuadraticSpline(Time(1:iCount),CO1i(1:iCount),Tini)
  CO2=QuadraticSpline(Time(1:iCount),CO2i(1:iCount),Tini)
EndIf                                              !ELJ

```

```

!PI Controllers
!*****
!Regulatory-Servo Control Implementation in Time-Domain
!*****
!Ziegler-Nichols; Cohen-Coon Tuning Methods
!*****
!Written by: Professor Michael Caracotsios and Elizabeth Jensen
!*****
Use AthenaUtilities
Global Tau11,Tau12,Tau21,Tau22,TauD1,TauD2,ThetaD1,ThetaD2 As Real
Global Thetap,Kp11,Kp12,Kp21,Kp22,KpD1,KpD2,Taud,Thetad As Real
Global Kd,Ku1,Ku2,Pu1,Pu2 As Real
Global Kc1,Kc2,Ti,Kcli,Kc2i,Td,Kcd,SP1,SP2,DS, SM, w, r, Ti2 As Real
Global Time(5000),Y1i(5000),Y2i(5000),Y3i(5000),Y4i(5000),Y5i(5000) As Real
Global Y6i(5000),CO1i(5000),CO2i(5000),PV1i(5000),PV2i(5000) As Real
Global E1i(5000),E2i(5000) As Real
Global iCount As Integer

w=1 !1=Ziegler-Nichols Ultimate Gain; 2=Ziegler- Nichols; 3=Cohen-Coon; ELJ
r=8! 8 = regulatory control; 9 = servo control; ELJ

!*****
!Process Gain for process as well as disturbance; ELJ
Kp11=0.1247 !ELJ
Kp12=0.546 !ELJ
Kp21=-0.1246 !ELJ
Kp22=1.116 !ELJ
KpD1=0.588 !ELJ
KpD2=1.41 !ELJ

!*****
!Process Lag for process as well as disturbance; ELJ
Tau11=10.0 !ELJ
Tau12=24.0 !ELJ
Tau21=16.0 !ELJ
Tau22=22.0 !ELJ
TauD1=22.0 !ELJ
TauD2=24.0 !ELJ

!*****
!Time Delay for both process and disturbance; ELJ
ThetaD1=7.0 !ELJ
ThetaD2=7.0 !ELJ
Thetap=5.0 !ELJ

!*****
!Ultimate Control Gains for process 1 and 2; ELJ
Ku1=30.577504 !Ultimate Gain for process 1; ELJ
Ku2=5.09088 !Ultimate Gain for process 2; ELJ

!*****
!Ultimate period for process 1 and 2; ELJ
Pu1=17.105 !Ultimate Period for process 1; ELJ
Pu2=17.975 !Ultimate Period for process 2; ELJ

SM=2.0 !Safety Margin; ELJ
Kc1=32.08 !Controller Gain for process 1; ELJ
Kc2=3.58 !Controller Gain for process 1; ELJ
Kcli=Kc1/4.0 !Integral Gain for control 1; ELJ
Kc2i=Kc2/4.0 !Integral Gain for control 2, ELJ
Kcd=0.0 !Derivative Gain !ELJ

!*****
!Ziegler-Nichols Ultimate Gain Controller Constants

```

```

! *****
!Proportional-Integral Controller
If(w==1)Then
    If(Kcli>0.0 .AND. Kcd==0.0)Then
        Kc1=0.45*Ku1/SM
        Ti=Pu1/1.2
        Kcli=Kc1/Ti
    EndIf

    If(Kc2i>0.0 .AND. Kcd==0.0)Then
        Kc2=0.45*Ku2/SM
        Ti2=Pu2/1.2
        Kc2i=Kc2/Ti2
    EndIf
EndIf

! *****
!Ziegler-Nichols Controller Constants
! *****
!Proportional-Integral Controller
If(w==2) Then
    If(Kcli>0.0 .AND. Kcd==0.0 )Then
        Kc1=(0.9/(SM*(Kp11)))*((Tau11)/ThetaD1)
        Ti=3.33*ThetaD1
        Kcli=Kc1/Ti
    EndIf

    If(Kc2i>0.0 .AND. Kcd==0.0 )Then
        Kc2=(0.9/(SM*(Kp22)))*((Tau22)/ThetaD2)
        Ti2=3.33*ThetaD2
        Kc2i=Kc2/Ti2
    EndIf
EndIf

! *****
!Cohen-Coon PD Controller Constants
! *****
!Proportional-Integral Controller
If(w==3) Then
    If(Kcli>0.0 .AND. Kcd==0.0)Then
        Kc1=(0.9/(SM*(Kp11)))*(((Tau11)/ThetaD1)+0.092)
        Ti=3.33*ThetaD1*(((Tau11)+0.092*ThetaD1)/((Tau11)+2.2*ThetaD1))
        Kcli=Kc1/Ti
    EndIf

    If(Kc2i>0.0 .AND. Kcd==0.0)Then
        Kc2=(0.9/(SM*(Kp22)))*(((Tau22)/ThetaD1)+0.092)
        Ti2=3.33*ThetaD2*(((Tau22)+0.092*ThetaD2)/((Tau22)+2.2*ThetaD2))
        Kc2i=Kc2/Ti2
    EndIf
EndIf

! *****
If(r==9)Then
    iCount=0
    SP1=0.02
    SP2=0.0
    DS=0.0
EndIf

! *****
If(r==8)Then
    iCount=0
    SP1=0.0
    SP2=0.0
    DS=0.1

```

```

EndIf

! *****
@Initial Conditions
U(1)=0.0 !ELJ
U(2)=0.0 !ELJ
U(3)=0.0 !ELJ
U(4)=0.0 !ELJ
U(5)=0.0 !ELJ
U(6)=0.0 !ELJ

! *****
@Model Equations
Dim PV1,PV2,CO1,CO2,IE As Real !ELJ
Dim i As Integer !ELJ

! *****
!Collect the History of the Integrated Model Variables
If (iRes==9) Then !ELJ
    iCount=iCount+1.0 !ELJ
    Time(iCount)=T !ELJ
    Y1i(iCount)=U(1) !ELJ
    Y2i(iCount)=U(2) !ELJ
    Y3i(iCount)=U(3) !ELJ
    Y4i(iCount)=U(4) !ELJ
    Y5i(iCount)=U(5) !ELJ
    Y6i(iCount)=U(6) !ELJ
EndIf !ELJ

! *****
!Formulate the Feedback Control Loop Equations
Select Case(iRange) !ELJ

    Case(0) !ELJ
        PV1=U(1)+U(2) !ELJ
        If (iRes==9) E1i(iCount)=SP1-PV1 !ELJ
        CO1=Kc1*(SP1-PV1)+Kcli*(SP1-PV1)*T !ELJ

        PV2=U(3)+U(4) !ELJ
        If (iRes==9) E2i(iCount)=SP2-PV2 !ELJ
        CO2=Kc2*(SP2-PV2)+Kcli*(SP2-PV2)*T !ELJ

        F(1)=0.0 !ELJ
        F(2)=0.0 !ELJ
        F(3)=0.0 !ELJ
        F(4)=0.0 !ELJ
        F(5)=0.0 !ELJ
        F(6)=0.0 !ELJ

    Case(1) !ELJ
        PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-
5)+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-5)
        If (iRes==9) E1i(iCount)=SP1-PV1 !ELJ
        IE=0.0 !ELJ
        Do i=1,iCount-1 !ELJ
            IE=IE+(E1i(i+1)+E1i(i))*(Time(i+1)-Time(i))/2.0 !ELJ
        EndDo
        IE=IE+(E1i(iCount)+SP1-PV1)*(T-Time(iCount))/2.0 !ELJ
        CO1=Kc1*(SP1-PV1)+Kcli*IE !ELJ
        PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-
5)+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-5)

        If (iRes==9) E2i(iCount)=SP2-PV2 !ELJ
        IE=0.0 !ELJ
        Do i=1,iCount-1 !ELJ
            IE=IE+(E2i(i+1)+E2i(i))*(Time(i+1)-Time(i))/2.0 !ELJ
        EndDo !ELJ

```

```

    IE=IE+(E2i(iCount)+SP2-PV2)*(T-Time(iCount))/2.0    !ELJ
    CO2=Kc2*(SP2-PV2)+Kc2i*IE                            !ELJ

    F(1)=-U(1)+Kp11*CO1                                  !ELJ
    F(2)=-U(2)+Kp12*CO2                                  !ELJ
    F(3)=-U(3)+Kp21*CO1                                  !ELJ
    F(4)=-U(4)+Kp22*CO2                                  !ELJ
    F(5)=0.0                                              !ELJ
    F(6)=0.0                                              !ELJ

    Case(2)                                              !ELJ
    PV1=QuadraticSpline(Time(1:iCount),Y1i(1:iCount),T-
5)+QuadraticSpline(Time(1:iCount),Y2i(1:iCount),T-
5)+QuadraticSpline(Time(1:iCount),Y5i(1:iCount),T-7)
    If(iRes==9)E1i(iCount)=SP1-PV1                      !ELJ
    IE=0.0                                              !ELJ
    Do i=1,iCount-1                                     !ELJ
    IE=IE+(E1i(i+1)+E1i(i))*(Time(i+1)-Time(i))/2.0    !ELJ
    EndDo                                              !ELJ
    IE=IE+(E1i(iCount)+SP1-PV1)*(T-Time(iCount))/2.0    !ELJ
    CO1=Kc1*(SP1-PV1)+Kc1i*IE                          !ELJ
    PV2=QuadraticSpline(Time(1:iCount),Y3i(1:iCount),T-
5)+QuadraticSpline(Time(1:iCount),Y4i(1:iCount),T-
5)+QuadraticSpline(Time(1:iCount),Y6i(1:iCount),T-7)    !ELJ

    If(iRes==9)E2i(iCount)=SP2-PV2                      !ELJ
    IE=0.0                                              !ELJ
    Do i=1,iCount-1                                     !ELJ
    IE=IE+(E2i(i+1)+E2i(i))*(Time(i+1)-Time(i))/2.0    !ELJ
    EndDo                                              !ELJ
    IE=IE+(E2i(iCount)+SP2-PV2)*(T-Time(iCount))/2.0    !ELJ
    CO2=Kc2*(SP2-PV2)+Kc2i*IE                          !ELJ

    F(1)=-U(1)+Kp11*CO1                                  !ELJ
    F(2)=-U(2)+Kp12*CO2                                  !ELJ
    F(3)=-U(3)+Kp21*CO1                                  !ELJ
    F(4)=-U(4)+Kp22*CO2                                  !ELJ
    F(5)=-U(5)+KpD1*DS                                  !ELJ

    F(6)=-U(6)+KpD2*DS                                  !ELJ

End Select                                              !ELJ

! *****
!Collect History of the Process Variables and Control Output

If (iRes==9)Then                                       !ELJ
    PV1i(iCount)=PV1                                  !ELJ
    PV2i(iCount)=PV2                                  !ELJ
    CO1i(iCount)=CO1                                  !ELJ
    CO2i(iCount)=CO2                                  !ELJ
EndIf                                                  !ELJ

! *****
@Coefficient Matrix
    E(1)=Tau11                                          !ELJ
    E(2)=Tau12                                          !ELJ
    E(3)=Tau21                                          !ELJ
    E(4)=Tau22                                          !ELJ
    E(5)=TauD1                                          !ELJ
    E(6)=TauD2                                          !ELJ

! *****
@Solver Options
    Neq=6                                              !ELJ
    Npts=75.0                                          !ELJ

```



```

Tend=250.0                                !ELJ
Tstop=5.0;7.0                             !ELJ

Param(28)=1      !Continue Integration Past Dead Time; ELJ
Param(177)=1      !Enable Integration History Access; ELJ
VariableNames=Time;Y1;Y2;Y3;Y4;Y5;Y6 !Variable Names; ELJ
Append=Pv1;Pv2    !Display Process Variable; ELJ

! *****
@After Calling Solver
Dim Pv1,Pv2,CO1,CO2 As Real                !ELJ
If(iCount==0)Then                          !ELJ
    Pv1=U(1)+U(2)+U(5)                     !ELJ
    Pv2=U(3)+U(4)+U(6)                     !ELJ
    CO1=Kc1*(SP1-Pv1)                       !ELJ
    CO2=Kc2*(SP2-Pv2)                       !ELJ
Else                                        !ELJ
    Pv1=QuadraticSpline(Time(1:iCount),Pv1i(1:iCount),Tini) !ELJ
    Pv2=QuadraticSpline(Time(1:iCount),Pv2i(1:iCount),Tini) !ELJ
    CO1=QuadraticSpline(Time(1:iCount),CO1i(1:iCount),Tini) !ELJ
    CO2=QuadraticSpline(Time(1:iCount),CO2i(1:iCount),Tini) !ELJ
EndIf                                       !ELJ

```